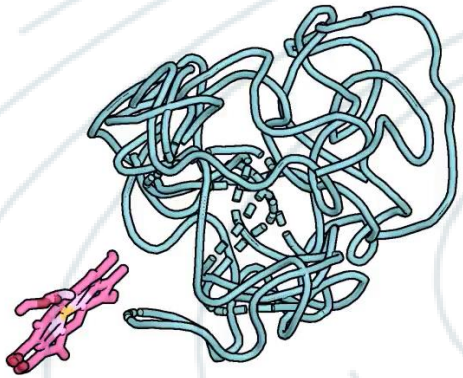
Continuous-Time Generative Modelling & Chemistry

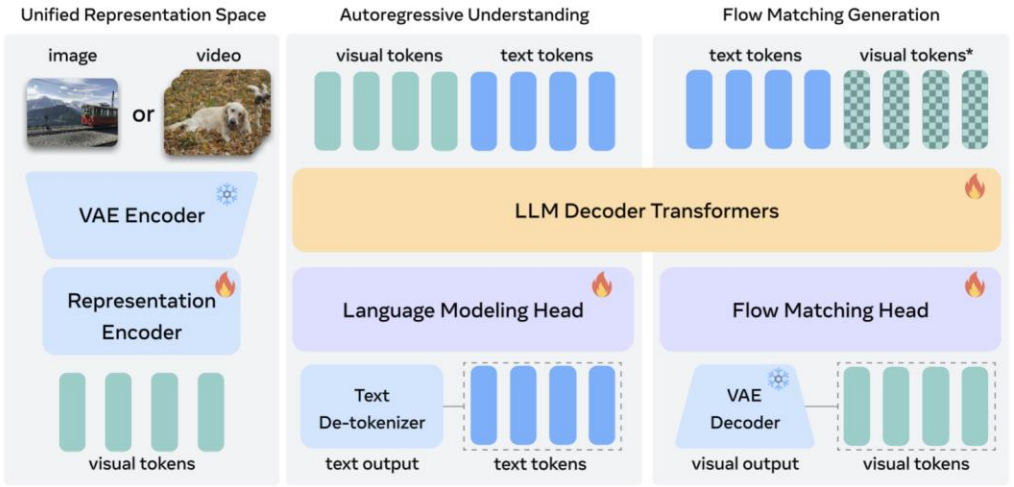
Chenghao Liu
Caltech

LASSLGRRIQLRGINKLTDNLTSFKEGSGPPNWELRNKAQGDIHNEKENAEAHKDLDGRDRFGPRDWNKDKVPEG
GTVVNITEGAGKAFYRPEGYTDWKQLILDWCCNVRGASGDLNGSHGLSAGPFWVVLKIVSCNVKERKRKVGVAVE

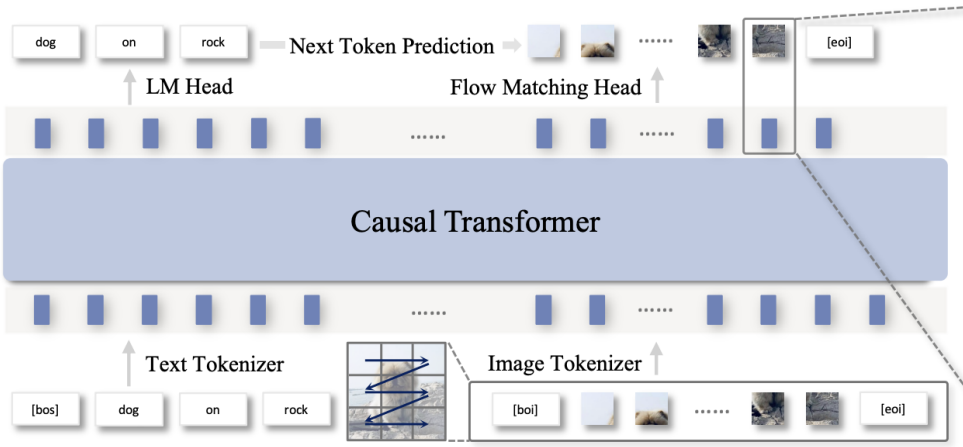


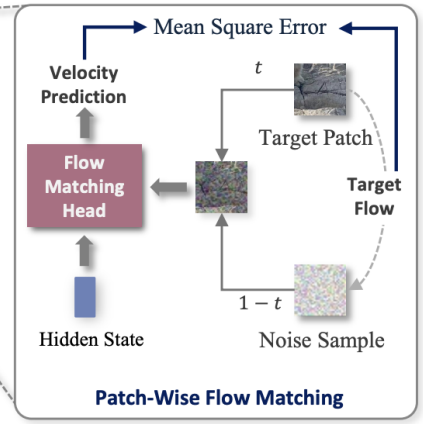
Slides partially developed with Alex Tong

Modern Image & Video Generation



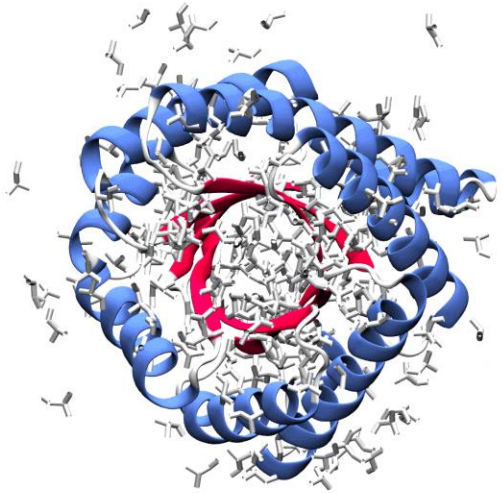
Wan





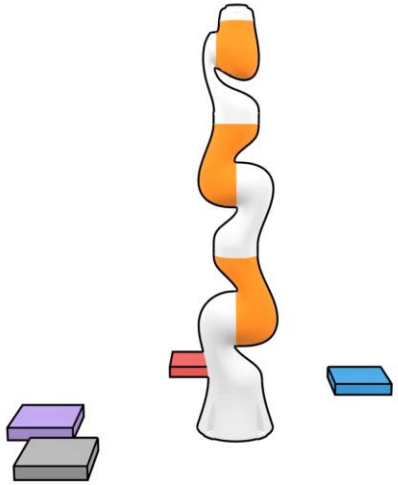
Beyond Images and Text

Scientific Data



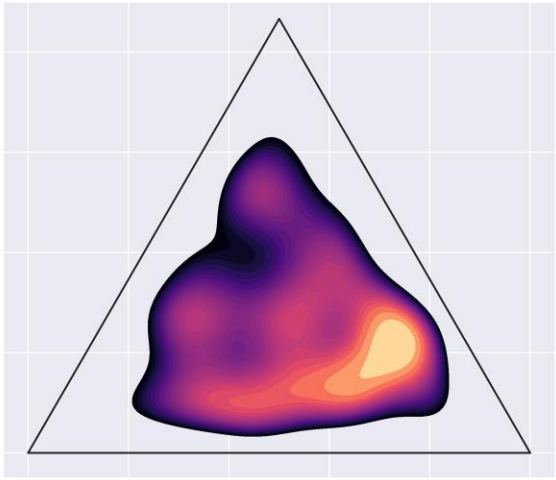
SE(3) invariant
Protein structure generation

Robotics



SO(2) invariant
Block stacking

Information Geometry



Fisher-Rao geometry
On the probability Simplex

Climate Modeling

Spherical Geometry $\mathbb{S}^2$

Outline

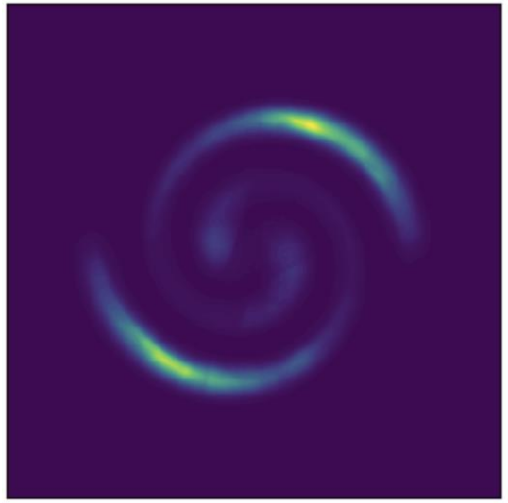
Part I. Simulation(-free) Generative Models

Part II. Fine-tuning & Steering

Part III. Applications to Chemistry

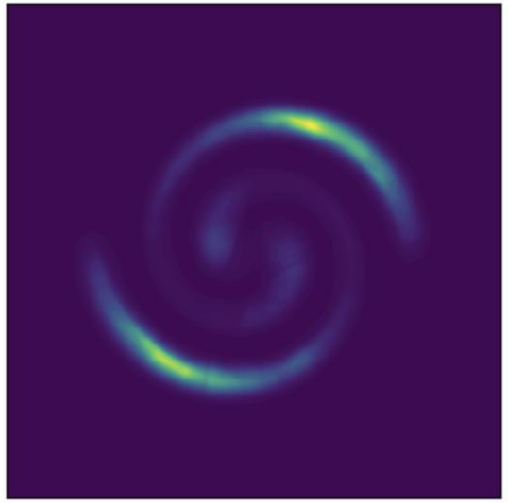
Problem Setting: Generative Modeling

- Unknown: data distribution q



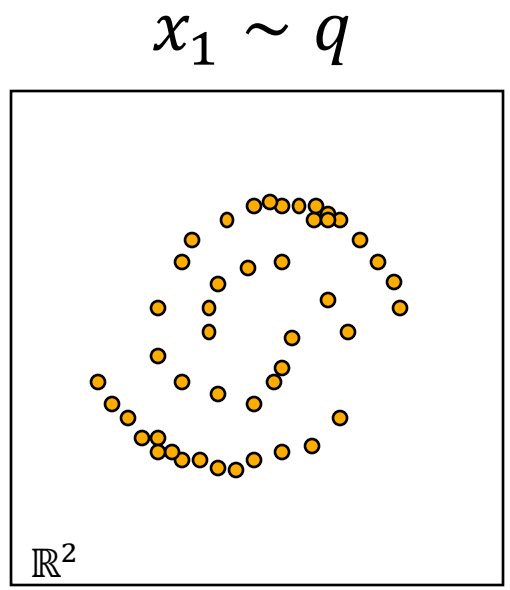
Problem Setting: Generative Modeling

- Unknown: data distribution q
- Given: samples $x_1 \sim q$



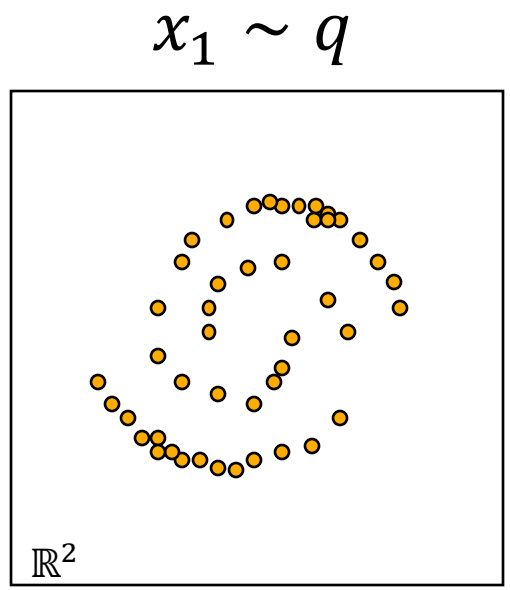
Problem Setting: Generative Modeling

- Unknown: data distribution q
- Given: samples $x_1 \sim q$



Problem Setting: Generative Modeling

- Unknown: data distribution q
- Given: samples $x_1 \sim q$

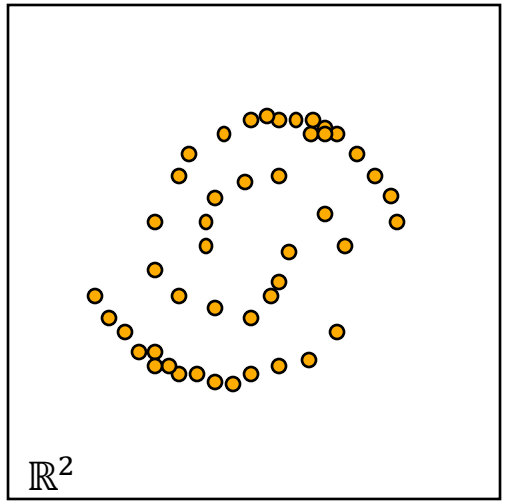


Goal: learn *a sampler* from the unknown q

Problem Setting: Generative Modeling

- Unknown: data distribution q
- Given: samples $x_1 \sim q$
- Learn: neural network with parameters θ

$x_1 \sim q$



Generative Model

(ψ_θ, p_θ)

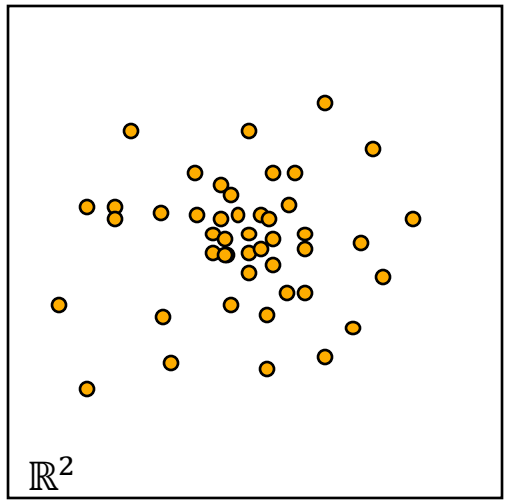
Generator Underlying Density

Goal: find parameters θ s.t. $p_\theta \approx q$

Deep Generative Modelling

Easy to sample from

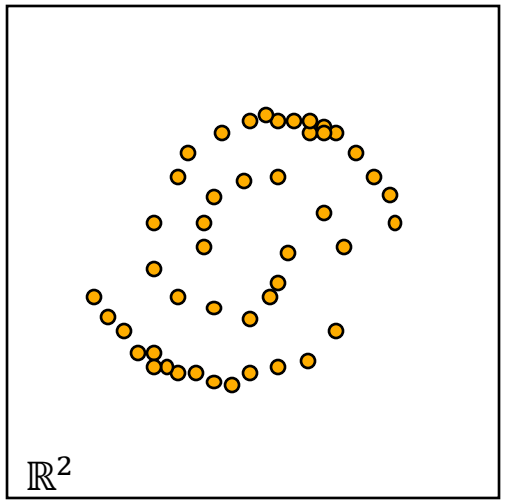
$x_0 \sim p$



Generator

ψ_θ

$x_1 \sim q$

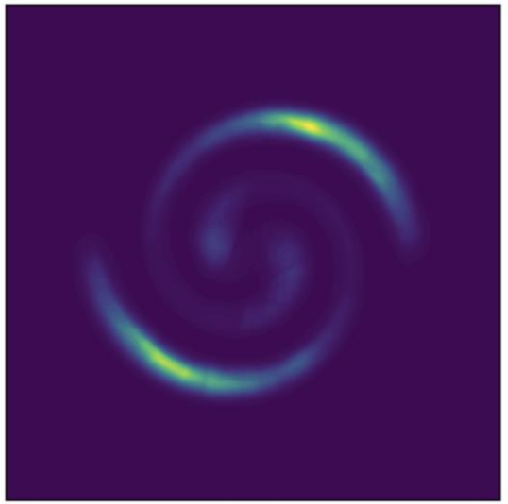


Sampling

$x_0 \sim p$

$\psi_\theta(x_0) \sim q$

How to model ψ_θ?



Density Estimation

$p_\theta \approx q$

Deep Generative Modelling

Fidelity

Samples should look like they came from q

Coverage

Model all of q, not just its easy modes, and ideally give log p(x)

Speed

Train fast (& stable). Call a few times during inference.

Deep Generative Modelling

1

One-step maps

GAN, VAE, Normalizing flows: one big jump from noise to data.

2

Neural ODEs

ODEs: infinitely many infinitesimal steps.

3

Diffusion

Add noise gradually; learn to undo it. It works — at a price.

4

Flow matching

Choose the path; Regress its velocity. No simulation.

5

One step, again

Learn the journey at once: consistency & flow maps.

Summary of Modern Deep Generative Models

Generative Model

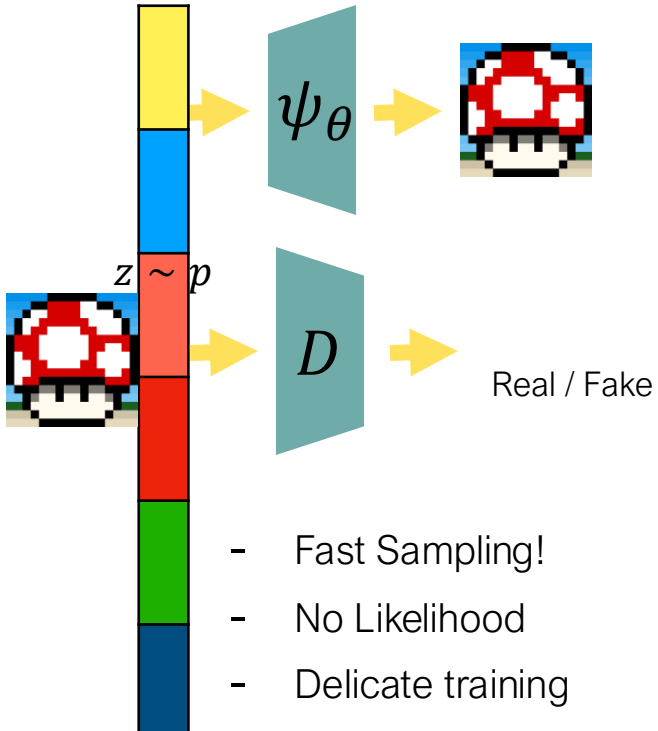
(ψ_θ, p_θ)

Autoregressive Models

$$p_\theta(x) = \prod_{i=1}^{d} p_\theta(x_i | x_{<i})$$

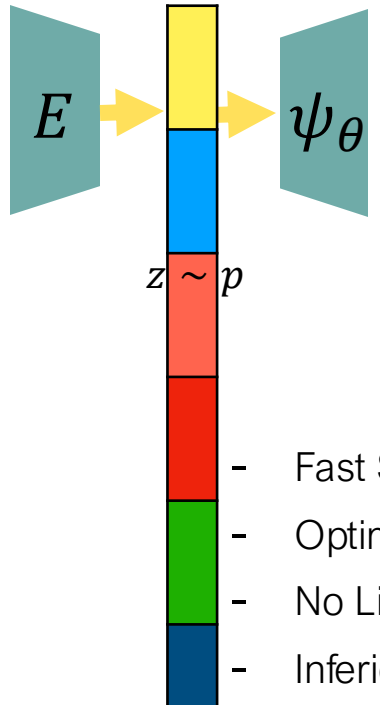
- Likelihood-based
- Sequential generation — scales with dimension
- SOTA in language modeling

GANs



- Fast Sampling!
- No Likelihood
- Delicate training

VAEs



- Fast Sampling!
- Optimize ELBO
- No Likelihood
- Inferior performance

Why are diffusion and flow useful?

GANs are still competitive performance-wise even today

Class-Conditional ImageNet 512×512					
Model	FID↓	sFID↓	IS↑	Precision↑	Recall↑
BigGAN-deep [10]	8.43	8.13	177.90	**0.88**	0.29
StyleGAN-XL [57]	**2.41**	**4.06**	267.75	0.77	0.52
Mask-GIT [12]	7.32	-	156.0	-	-
ADM [19]	23.24	10.19	58.06	0.73	0.60
ADM-G, ADM-U	3.85	5.86	221.72	0.84	0.53
Simple Diffusion(U-Net) [28]	4.28	-	171.0	-	-
Simple Diffusion(U-ViT, L)	4.53	-	205.3	-	-
VDM++ [36]	2.65	-	278.1	-	-
DiT-XL(cfg = 1.5) [50]	3.04	5.02	240.82	0.84	0.54
SiT-XL(cfg = 1.5, SDE)	**2.62**	4.18	252.21	0.84	0.57

It's about training stability and inference flexibility

Normalizing Flows

Sample from some complicated distribution by sampling from a simple distribution then applying U

- Begin with a simple distribution

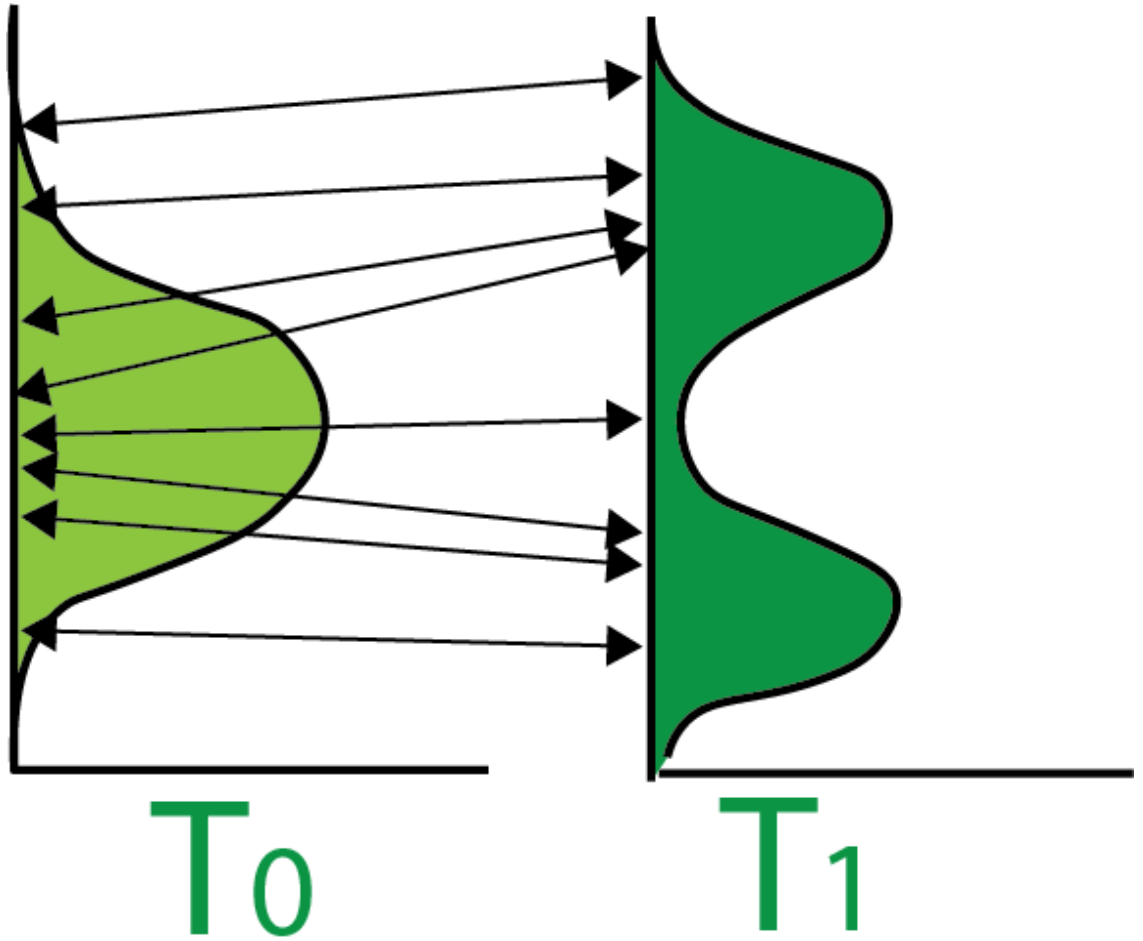
$$p_{t_0}(x) \sim \mathcal{N}(0, 1)$$

- Apply an *invertible* transformation(s)

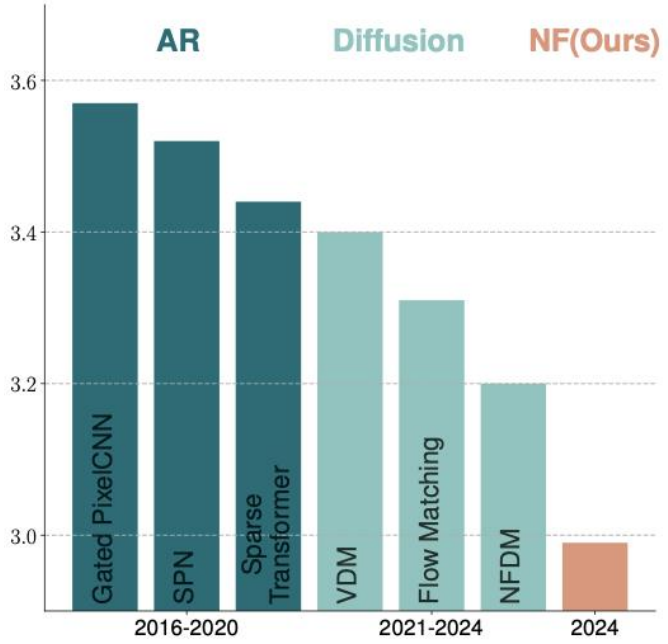
$$x_{t_1} = U(x_{t_0})$$

- Use change of variables to calculate probability

$$\log p_{t_1}(x_{t_1}) = \log p_{t_0}(x_{t_0}) - \log \det \left| \frac{\partial U}{\partial x_{t_0}} \right|$$



Normalizing Flows



Normalizing Flows are Capable Generative Models

Shuangfei Zhai [1] **Ruixiang Zhang** [1] **Preetum Nakkiran** [1] **David Berthelot** [1] **Jiatao Gu** [1] **Huangjie Zheng** [1]
Tianrong Chen [1] **Miguel Angel Bautista** [1] **Navdeep Jaitly** [1] **Josh Susskind** [1]

Deep Normalizing Flows

✓ exact log-likelihood
✓ sampling in one pass
✗ architecture restricted to invertible blocks
✗ latent dimension must equal data dimension

Apply a series of transformations

$$x_{t_1} = U(x_{t_0}) \rightarrow x_{t_N} = u_N \circ u_{N-1} \circ \cdots \circ u_1(x_{t_0})$$

Use change of variables to calculate probability

$$\log p_{t_1}(x_{t_1}) = \log p_{t_0}(x_{t_0}) - \log \det \left| \frac{\partial U}{\partial x_{t_0}} \right| \rightarrow \log p_{t_N}(x_{t_N}) = \log p_{t_0}(x_{t_0}) - \sum_{n=1}^{N} \log \det \left| \frac{\partial u_n}{\partial x_{t_{n-1}}} \right|$$

[Chen et al. 2018; Grathwohl et al. 2018]

Continuous Normalizing Flows

Infinitely many layers
= an ordinary differential equation

Apply a series of transformations

$$x_{t_N} = u_N \circ u_{N-1} \circ \cdots \circ u_1(x_{t_0}) \quad \rightarrow \quad x_{t_1} = U(x_{t_0}) = \int_{t_0}^{t_1} u(x(t), t)\mathrm{d}t$$

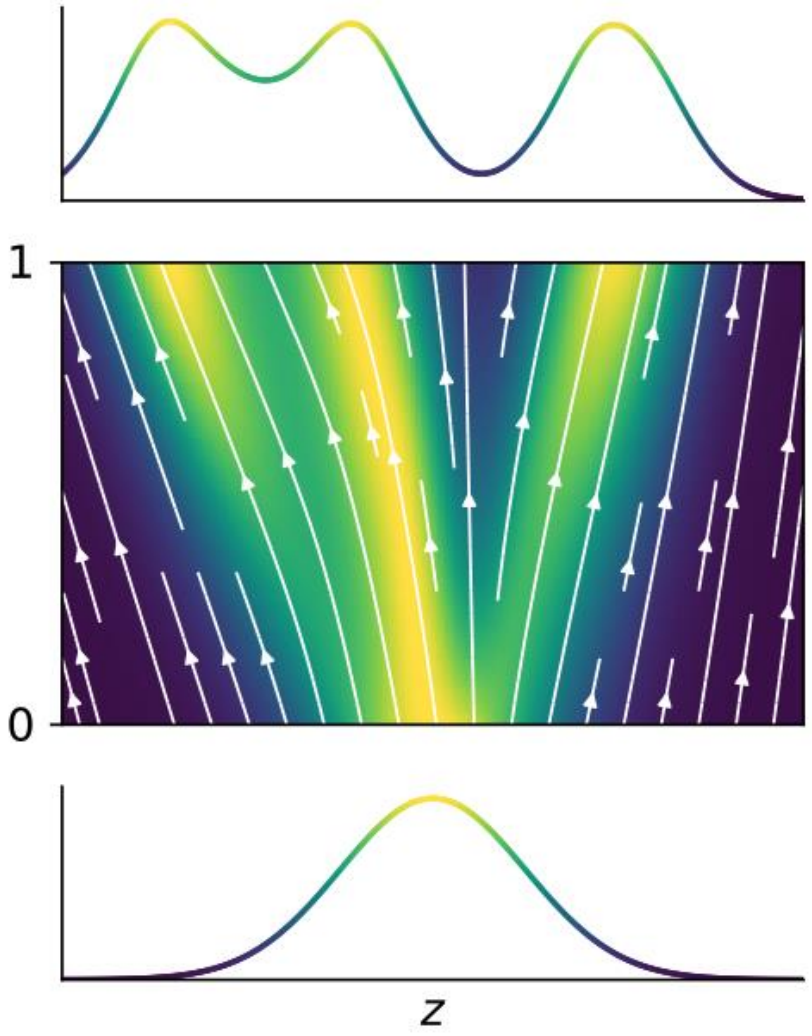
Use change of variables to calculate probability

$$\log p_{t_N}(x_{t_N}) = \log p_{t_0}(x_{t_0}) - \sum_{n=1}^{N} \log \det \left| \frac{\partial u_n}{\partial x_{t_{n-1}}} \right| \quad \rightarrow \quad \log p_{t_1}(x_{t_1}) = \log p_{t_0}(x_{t_0}) - \int_{t_0}^{t_1} \mathrm{Tr}\left(\frac{\partial u}{\partial x(t)} \right) \mathrm{d}t$$

Constructs an invertible transformation ψ for any f!

[Chen et al. 2018; Grathwohl et al. 2018]

The catch: training requires simulation



- To compute one training likelihood, you must solve the ODE for that sample — typically ~100 network evaluations…
- …while estimating the divergence along the way (Hutchinson trace estimator)…
- …and then backpropagate through the entire solve (or solve an adjoint ODE backwards).
- **Hundreds of network calls per gradient step.**

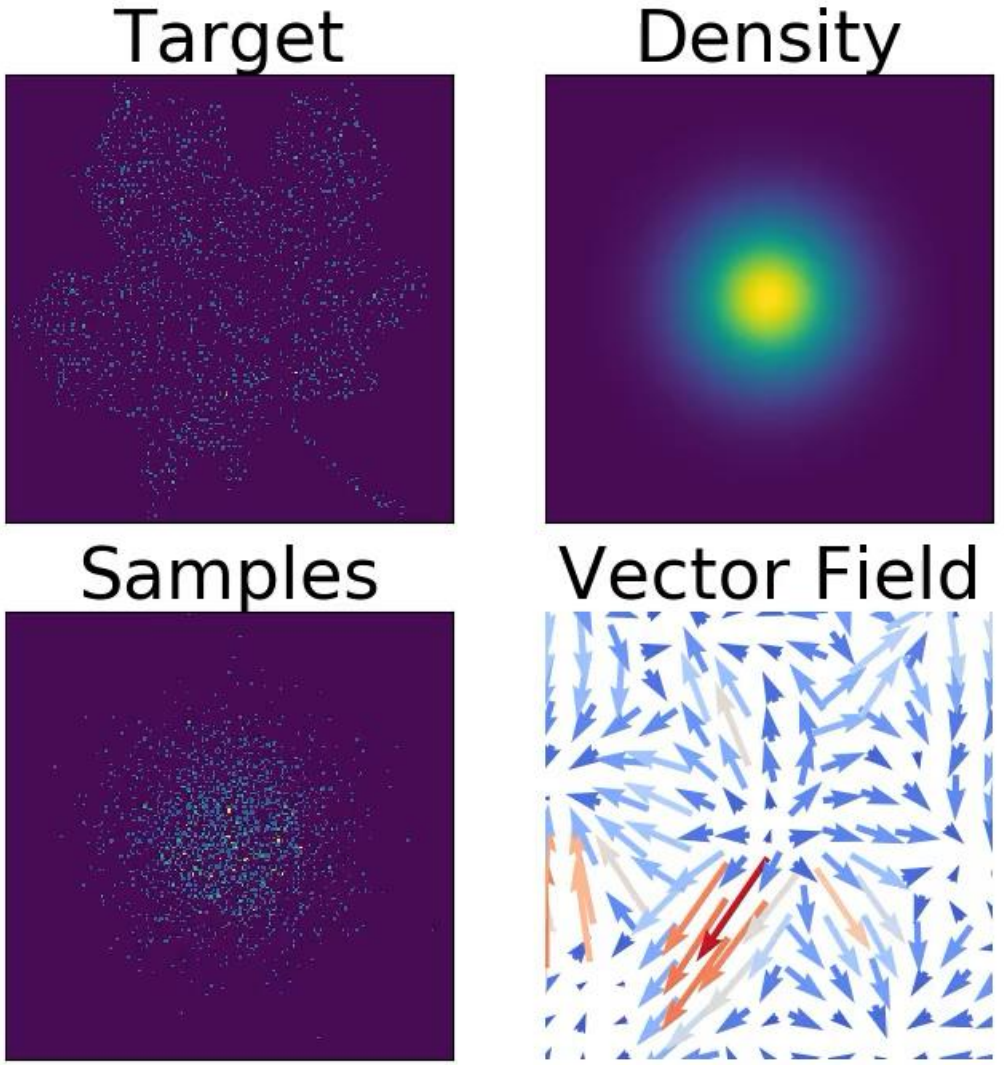
- **Can we not solve ODEs?**

Short symmetry

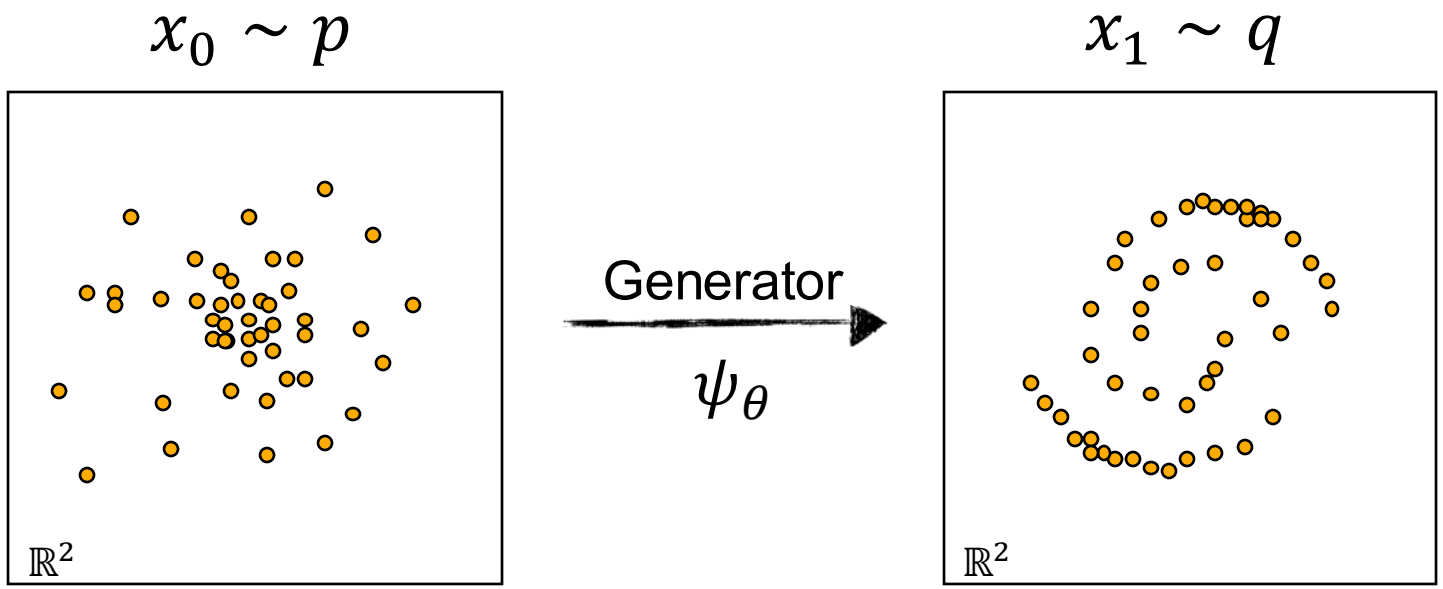
Discrete flows pay in architecture (constrained layers, cheap training).

Continuous flows pay in training time (free architecture, costly simulation).

How does this look like?

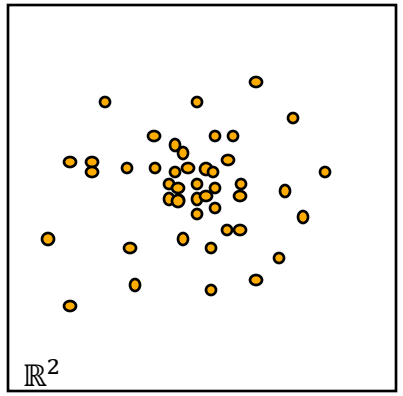


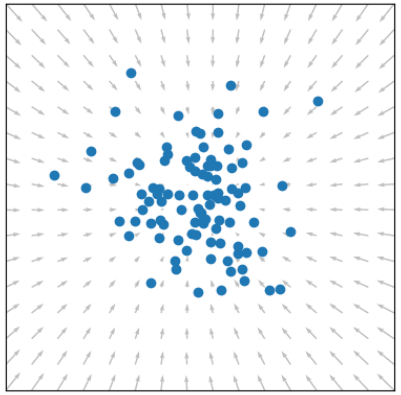
Generative Models as Dynamic Systems



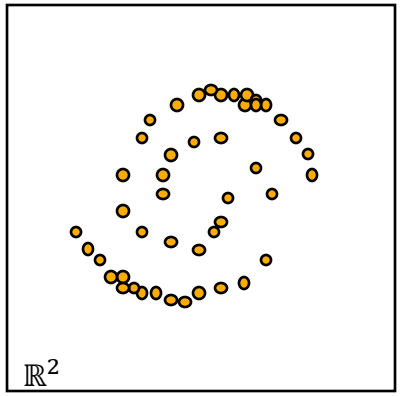
Dynamical Systems as Generative Models

$x_0 \sim p$





$x_1 \sim q$



$$\psi_t : [0,1] \times \mathbb{R}^2 \to \mathbb{R}^2$$

Time-Dependent Generator

Sampling = Simulating

$x_0 \sim p$ → $\text{simulate}(x_0, t) = \psi_t(x_0)$

Deterministic and Stochastic Dynamics

Flows

ODE $$dx_t = u_t(x_t)dt$$

Velocity field

Diffusion

SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Browniar Motion

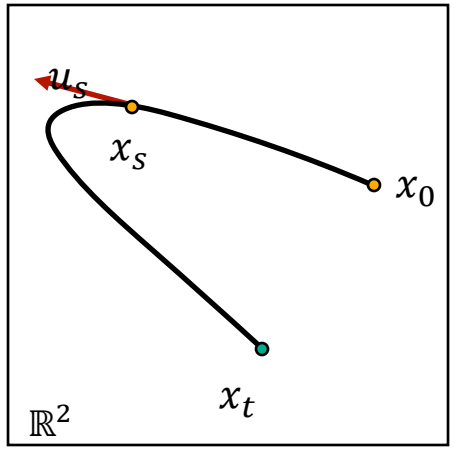
Deterministic and Stochastic Dynamics

Flows

ODE $dx_t = u_t(x_t)dt$

Velocity field

Deterministic



$x_t = x_0 + \int_0^t u_s(x_s)ds$

Diffusion

SDE $dx_t = f_t(x_t)dt + g_t dw_t$

Drift

Diffusion Coefficient

Browniar Motion

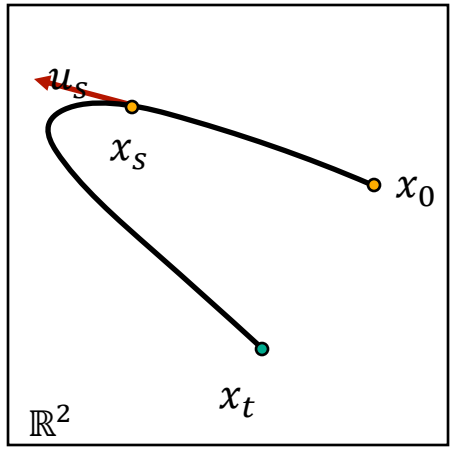
Deterministic and Stochastic Dynamics

Flows

ODE $$dx_t = u_t(x_t)dt$$

Velocity field

Deterministic



$$x_t = x_0 + \int_0^t u_s(x_s)ds$$

Diffusion

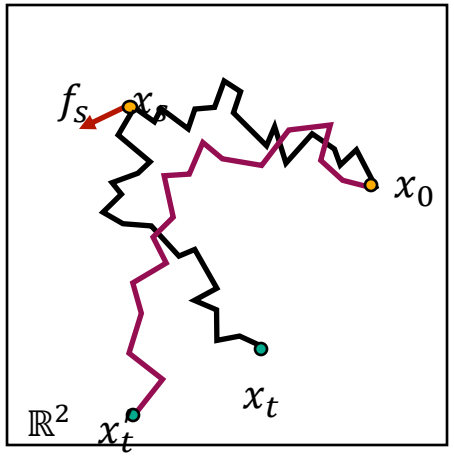
SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Browniar Motion

Stochastic



$$x_t = x_0 + \int_0^t f_s(x_s)ds + \int_0^t g_s dw_s$$

Where are my probabilities?

Flows

ODE $$dx_t = u_t(x_t)dt$$

Velocity field

The Continuity Equation

$$\partial_t p_t = -\mathrm{div}(p_t u_t)$$

Diffusion

SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Browniar Motion

The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Where are my probabilities?

Flows

ODE $dx_t = u_t(x_t)dt$

Velocity field

Diffusion

SDE $dx_t = f_t(x_t)dt + g_t dw_t$

Drift

Diffusion Coefficient

Browniar Motion

The Continuity Equation

$$\partial_t p_t = -\mathrm{div}(p_t u_t)$$

The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Yes!

Need one more thing…

Can we build a generative model with these?

Diffusion and score models

SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Brownian Motion

The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Need one more thing…

Diffusion and score models

Forward SDE $dx_t = f_t(x_t)dt + g_t dw_t$

Data → Noise

The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Need one more thing…

Diffusion and score models

Forward SDE $dx_t = f_t(x_t)dt + g_t dw_t$ Data → Noise

Reverse SDE $d\overline{x}_t = (f_t(x_t) - g_t^2 \nabla \log p_t)dt + g_t d\overline{w}_t$ Noise → Data

The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Need one more thing… The Score!

Diffusion and score models

Forward SDE $dx_t = f_t(x_t)dt + g_t dw_t$ Data → Noise

Reverse SDE $d\overline{x}_t = (f_t(x_t) - g_t^2 \nabla \log p_t)dt + g_t d\overline{w}_t$ Noise → Data

Learn the score by regressing to conditional scores:

$$\min_\theta \mathbb{E}_{p_{data}, p_t(x|x_{data})}\left[\| s_t^\theta(x) - \nabla \log p_t(x|x_{data}) \|^2\right]$$

Simulation-free

Known SDEs: Variance Exploding
Variance Preserving

Where are my probabilities?

Flows

ODE $$dx_t = u_t(x_t)dt$$

Velocity field

The Continuity Equation

$$\partial_t p_t = -\mathrm{div}(p_t u_t)$$

Diffusion

SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Browniar Motion

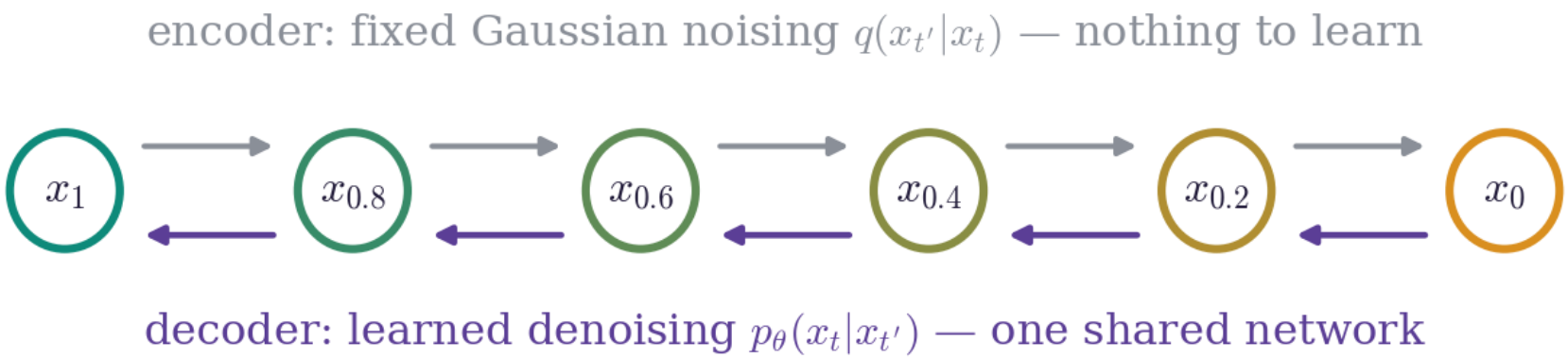
The Fokker-Planck Equation

$$\partial_t p_t = -\mathrm{div}(p_t f_t) + \frac{1}{2} g_t^2 \nabla^2 p_t$$

Learn: score $\nabla \log p_t$

- Only Gaussian source
- Solution asymptotically reaches source

Diffusion is a many-step VAE



- Latents = the noisy chain x_t. The encoder is the fixed noising process.
- The decoder undoes one small step at a time; each step is a nearly-Gaussian, nearly-linear denoising problem

Flow models

Flows

ODE $dx_t = u_t(x_t)dt$

Velocity field

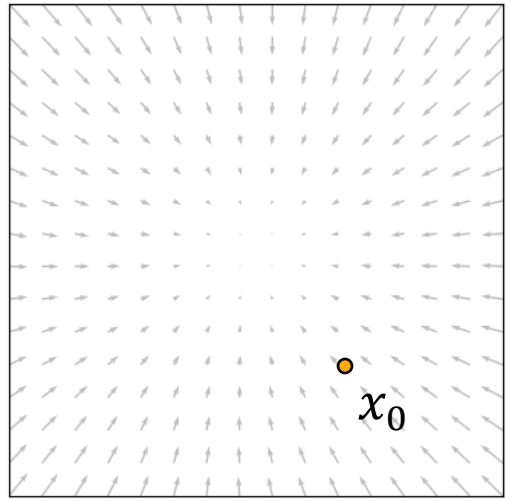
The Continuity Equation

$$\partial_t p_t = -\mathrm{div}(p_t u_t)$$

Yes!

Flow ODE

$$\dot{\psi}_t(x_0) = u_t(\psi_t(x_0))$$



$\psi_t(x)$ is smooth with smooth **inverse** defined by $-u_t(x)$

Where are my probabilities?

Flow ODE

$$\dot{\psi}_t(x_0) = u_t(\psi_t(x_0))$$

Diffusion

SDE $$dx_t = f_t(x_t)dt + g_t dw_t$$

Drift

Diffusion Coefficient

Browniar Motion

The Continuity Equation

$$\partial_t p_t = -\mathrm{div}(p_t u_t)$$

The Liouville Equation

$$\partial_t p_t = -\mathrm{div}(p_t(f_t - \frac{1}{2} g_t^2 \nabla \log p_t))$$

Learn: velocity field u_t

- Universal transformation between densities
- Defined on finite time interval

Learn: score $\nabla \log p_t$

- Only Gaussian source
- Solution asymptotically reaches source

Flow vs Flow matching models

Velocity field u_t is unknown: normally need simulation

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,\, x\sim p_t} \|v_\theta(x,\, t) - u_t(x)\|^2$$

$$\log p_1(x_1) = \log p(x_0) + \int_1^0 \mathrm{div}(u_t(x_t))dt$$

$$x_t = x_1 + \int_1^t u_s(x_s)ds$$

Hacks: condition on a single datapoint

Per pair (x_0, x_1), the path and its velocity are known in closed form:

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\, x_1,\, x\sim p_t(\cdot\,|\,x_1)} \|v_\theta(x,\, t) - u_t(x\,|\,x_1)\|^2$$

$$\nabla_\theta \mathcal{L}_{\mathrm{FM}} = \nabla_\theta \mathcal{L}_{\mathrm{CFM}}$$

Intuition: regress on single-pair velocities; the network's own averaging recovers the marginal field.

The simplest path is (nearly) the best one

Linear interpolation

Conditional path: a straight line, constant velocity

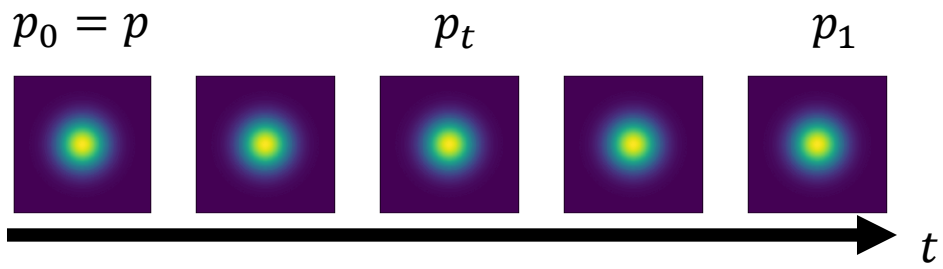
$$x_t = (1-t)\,x_0 + t\,x_1, \qquad u_t(x_t \mid x_0, x_1) = x_1 - x_0$$

The entire training loop

x0 = randn(); x1 = data(); t = rand()

xt = (1-t)*x0 + t*x1

loss = || v(xt, t) - (x1 - x0) ||²



Why straightness pays: Euler's error comes from curvature. Straighter marginal trajectories ⇒ fewer, larger steps at sampling time.

Flow vs diffusion

- For Gaussian paths $x_t = \alpha_t x_1 + \sigma_t x_0$, velocity and score are linearly interconvertible.
- They are almost always the same in practice, especially during training.
- The interesting question is whether to use noise during inference.
  - Deterministic is slightly faster
  - Stochastic is slightly better performance
- Two applications where it does matter
  - Manifolds, e.g. molecules
  - Sparse priors, e.g. image-to-image

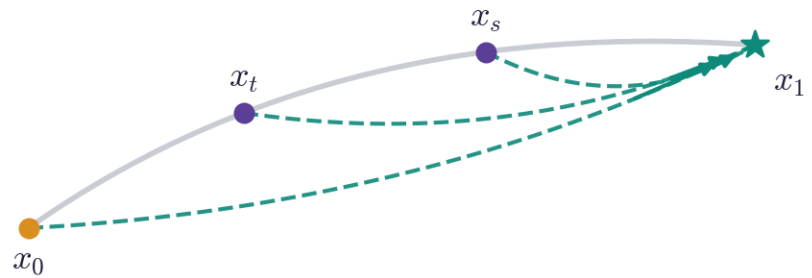
Consistency models: Fewer steps

$$f_\theta(x_t, t) \approx \Psi_{t\to 1}(x_t)$$

Self-consistency + boundary condition

$$f_\theta(x_t, t) = f_\theta(x_s, s), \qquad f_\theta(x_1, 1) = x_1$$

The trick: Network returns same value for any two points in the trajectory, and at t=1 it returns the data. Agreement between every neighboring pair chains into agreement between all pairs -> few steps.



f_θ maps every point on a trajectory to its endpoint

Train by consistency distillation (from a pretrained field) or consistency training (from scratch).

Consistency Models

Yang Song [1] Prafulla Dhariwal [1] Mark Chen [1] Ilya Sutskever [1]

ONE STEP DIFFUSION VIA SHORTCUT MODELS

Kevin Frans
UC Berkeley
kvfrans@berkeley.edu

Danijar Hafner
UC Berkeley

Sergey Levine
UC Berkeley

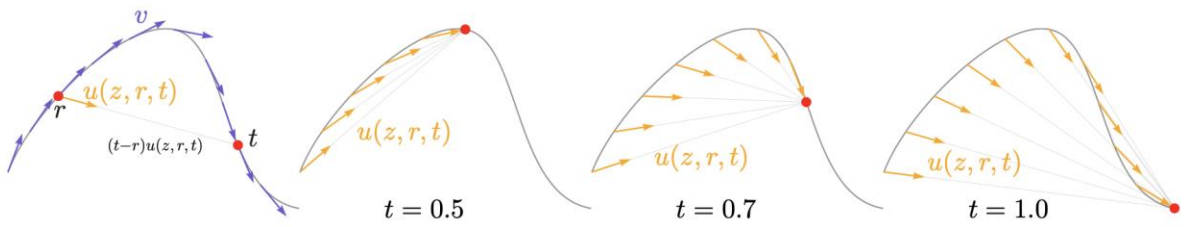
Pieter Abbeel
UC Berkeley

Mean Flows

Mean Flows for One-step Generative Modeling

Zhengyang Geng[1]* Mingyang Deng[2] Xingjian Bai[2] J. Zico Kolter[1] Kaiming He[2]

[1]CMU [2]MIT



Idea:
Condition on current and target time!

$$u_\theta(x_t, t)$$

$$u_\theta(x_t, r, t)$$

How to learn $u_\theta(x_t, r, t)$

$$(t-r)u(x_t, r, t) = \int_r^t v(x_t, \tau)d\tau$$

Derivative w.r.t. t

$$u(x_t, r, t) + (t-r)\frac{d}{dt}u(x_t, r, t) = v(x_t, t)$$

Rearrange

$$u_{target}(x_t, r, t) = v(x_t, t) - (t-r)\frac{d}{dt}u(x_t, r, t)$$

$$L(\theta) = min\mathbb{E}_{t,q(z),p_t(x|z)} \parallel u_t^\theta(x, r, t) - stopgrad(u_{target}) \parallel^2$$

How to learn $u_\theta(x_t, r, t)$

$$(t-r)u(x_t,r,t) = \int_r^t v(x_$$

Derivative w.r.t. t

$$u(x_t,r,t) + (t-r)\frac{d}{dt}u(x_t,r,t) = v(x_t$$

Rearrange

$$u_{target}(x_t,r,t) = v(x_t,t$$

$$L(\theta) = min\mathbb{E}_{t,q(z),p_t(x|z)} \parallel u_t^\theta(x,r,t) - stopgrad(u_{target}) \parallel^2$$

Algorithm 1 MeanFlow: Training.

Note: in PyTorch and JAX, `jvp` returns the function output and JVP.

```
# fn(z, r, t): function to predict u
# x: training batch

t, r = sample_t_r()
e = randn_like(x)

z = (1 - t) * x + t * e
v = e - x

u, dudt = jvp(fn, (z, r, t), (v, 0, 1))

u_tgt = v - (t - r) * dudt
error = u - stopgrad(u_tgt)

loss = metric(error)
```

Algorithm 2 MeanFlow: 1-step Sampling

```
e = randn(x_shape)
x = e - fn(e, r=0, t=1)
```

Summary

Model	The map	Trained by	Exact log p?	Steps	Main constraint
Normalizing flow	invertible network	exact max likelihood	yes	1	invertibility requirement
VAE	stochastic decoder	ELBO (bound)	hard	1	one giant leap
GAN	free-form network	adversarial game	no	1	instability; mode dropping
Autoregressive	factorized sampling	exact max likelihood	yes	d (per dim)	sequential generation
CNF	ODE solution	exact ML via simulation	yes	many	simulation inside training
Diffusion	SDE / PF-ODE	denoising regression (= ELBO)	hard	10–100+	Gaussian path; many steps
Flow matching	ODE solution	velocity regression (CFM)	hard	5–50	Many steps
Flow map	trajectory endpoint map	self-consistency / MF identity	hard	1–4	Scaling & stability

Take-aways

- Gotta have samples!
- Generative modelling is a coupling problem: how to best map noise onto data?
- Rich training signal & inference choices drive performance.
- Flow and diffusion are basically the same.
- Most useful due to stability & flexibility.
- Many small tricks for ‘best’ performance.

Unsolved challenges:

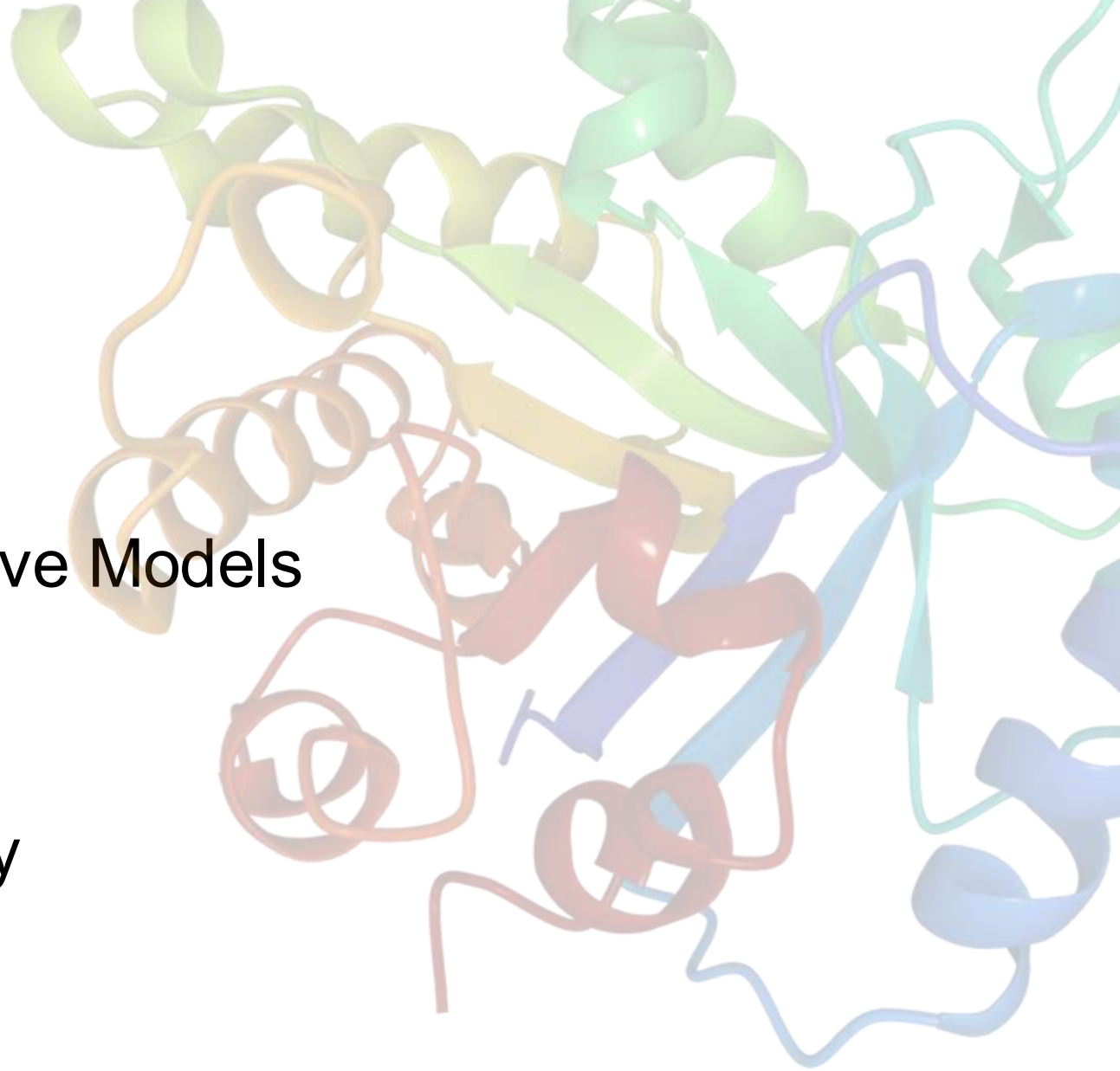
- Discrete space: Autoregressive remains the best model. How to do few step generation? How to scale diffusion model (can they be scaled?)
- Continuous space: How to do multimodal generation well, especially few step?

Outline

Part I. Simulation(-free) Generative Models

Part II. Fine-tuning & Steering

Part III. Applications to Chemistry



Pretrained models & the desired outputs

sans fine-tuning

package
package
package
package
package

By 2015-06-03 14:51:15.87945 WhyThisPost
By 2015-06-03 14:53:13.53923 WhatThisPost
By 2015-06-03 14:54:29.25974 WhenThis

```
```
```
```
```

*with fine-tuning*

In the quiet corners of the world, where the hum of daily life fades into the background, there exists a space where imagination and reality intertwine. It is a place where stories are not just told but felt, where every word carries the weight of possibility. Whether through the pages of a book, the brushstrokes of a painting, or the melodies of a song, this space invites us to explore, to dream, and to connect with something greater than ourselves

Pretraining gives a model

$$p_\theta \approx p_{data}$$
```

Moving from p_θ to desired outputs

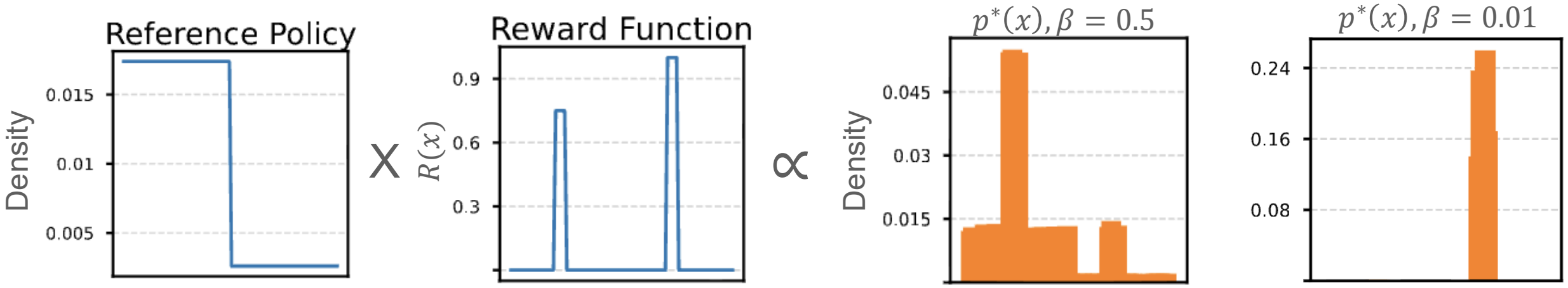
$$Z = \int p_{\theta,ref}(x) \exp\left(\frac{R(x)}{\beta}\right)$$

Reward

$R(x)$

Reward tilted distribution:

$$p^*(x) = \frac{1}{Z} p_{\theta,ref}(x) \exp\left(\frac{R(x)}{\beta}\right)$$



3 ways to sample from the target

Offline tuning

Train on some nice examples

Online tuning

Reinforcement learning to sample from the reward

Inference steering

No training, just adapt the sampling procedure

All sample from the *reward tilted* distribution:

$$p^*(x) \propto p_{\theta,ref}(x) \exp\left(\frac{R(x)}{\beta}\right)$$

Offline: Supervised finetuning

$$KL(p \parallel q) = \mathbb{E}_{x\sim p}\left[\log \frac{p(x)}{q(x)}\right]$$

Objective:

$$\max_{\theta} \mathbb{E}_{x\sim\mathcal{D}}[\log p_\theta(x)] \Leftrightarrow \min_{\theta} KL(p_{\mathcal{D}} \| p_\theta)$$

Sample base model

Generate n samples from $p_{\theta,ref}(x)$

Filter by reward

Keep top k samples

SFT on good samples

Retrain $p_{\theta,ref}(x)$ on good samples

Key insight: as N → ∞, Best-of-N converges to $p^*(x)$

$$\propto p_{ref}(x)\, exp\left(\frac{R(x)}{\beta}\right)!$$

Broadly used, but might mode collapse!

Ouyang, Long, et al. "Training language models to follow instructions with human feedback." *NeurIPS* (2022).

Offline: Direct Preferential Optimization

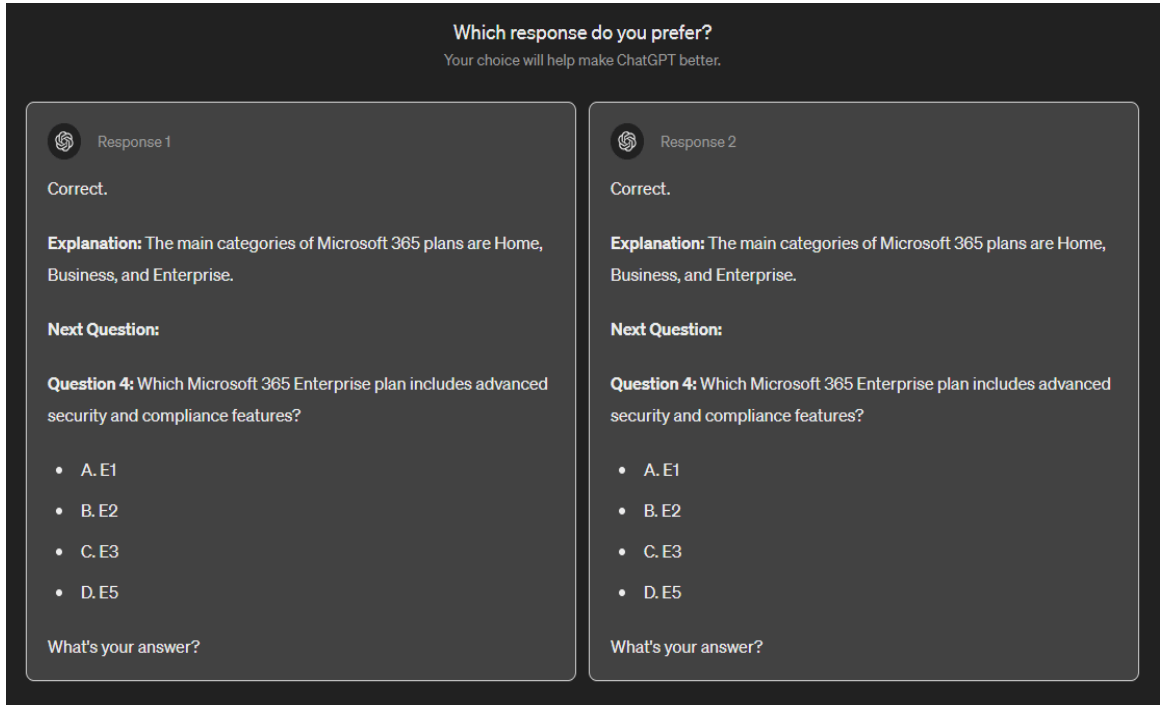
Dataset of pairs

$$\mathcal{D} = \left\{ \left(x_1^{(i)}, x_2^{(i)} \right) \middle| \; x_1^{(i)} \text{ is “better than” } x_2^{(i)} \right\}$$

Assume a specific form of the optima (Bradley-Terry), then your *reward is given by your model!*

$$R(x) = \beta \log \frac{p_\theta(x)}{p_{ref}(x)} + \log Z$$

Maximum likelihood training with $\mathcal{D}$ and end up sampling from $p^*(x)$



Hard to explore

Wallace, Bram, et al. "Diffusion model alignment using direct preference optimization." *CVPR* (2024).

Offline: Direct Preferential Optimization

$$L_{DPO}(p_\theta, p_{ref}) = -\mathbb{E}_{(x_1,x_2)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{p_\theta(x_1)}{p_{ref}(x_1)} - \beta\log\frac{p_\theta(x_2)}{p_{ref}(x_2)}\right)\right]$$

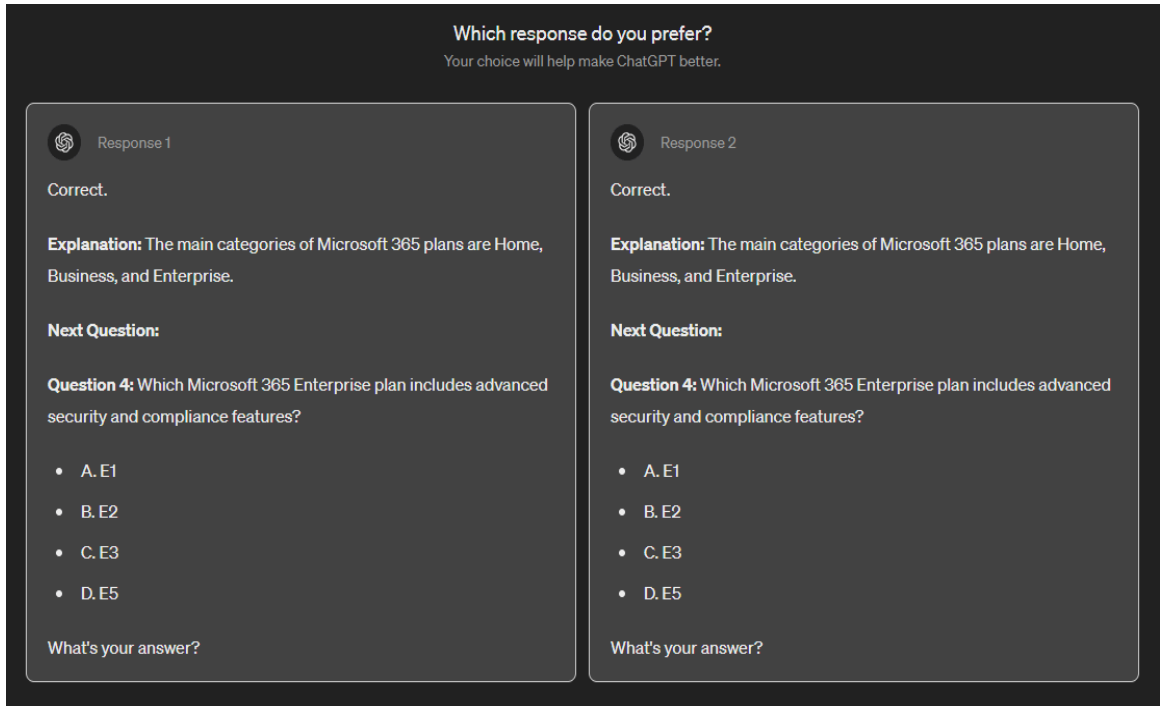
Easy for autoregressive

$$p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i|x_1, \ldots, x_{i-1})$$

$$p(\text{The cat sat}) = p(\text{The})p(\text{cat}|\text{The})p(\text{sat}|\text{The cat})$$

Hard for diffusion

$$p(x_0) = \int p(x_{0:T})dx_{1:T}$$

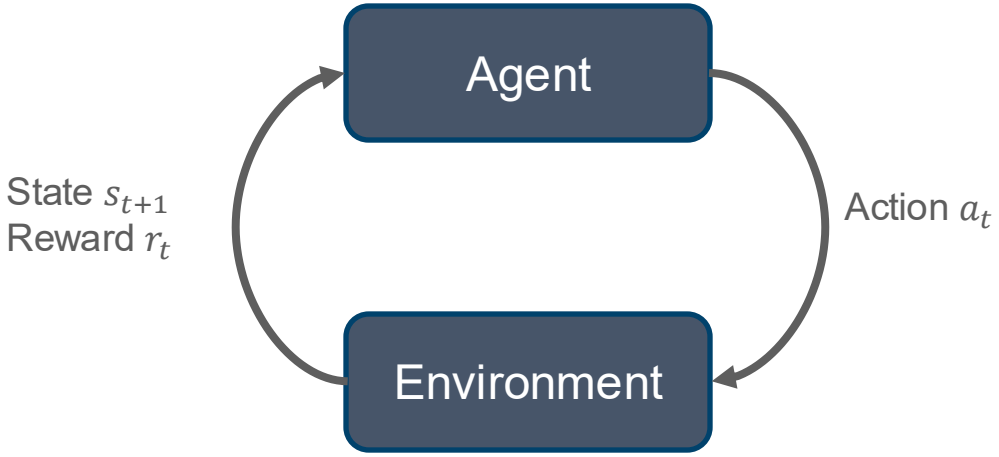


Hard to explore

Wallace, Bram, et al. "Diffusion model alignment using direct preference optimization." *CVPR* (2024).

Online: Reinforcement Learning

Agent interacts with an **environment** that provides a **reward**



Goal: Learn the policy the agent should follow that will maximize the agent's reward

$$p^*_{RL} = \underset{p}{argmax}\, \mathbb{E}_{x_{1:T}\sim p}\left[\sum_{i=1}^{T} R(x_i)\right]$$

Online: Reinforcement Learning

Train with a KL regularized version of the RL objective

$$\max_{\theta} \mathbb{E}_{x \sim p_\theta(x)}[R(x)] - \beta \cdot KL(p_\theta \parallel p_{ref})$$

Good reward *Don't go too far*

The KL regularized RL objective converges to the reward tilted distribution!

$$p^*(x) = \frac{1}{Z} p_{ref}(x) \exp\left(\frac{R(x)}{\beta}\right)$$

Korbak, Tomasz, Ethan Perez, and Christopher Buckley. "RL with KL penalties is better viewed as Bayesian inference." *EMNLP (2022)*.

Online: Getting reward the old way, RLHF

$$\min_{\phi} \mathbb{E}_{(x\ y)\sim\mathcal{D}} \left[\left\| R_\phi(x) - y \right\|_2^2 \right]$$

Reward model

Init from base model, fine-tune a scalar head (e.g. human preference)

Retrained

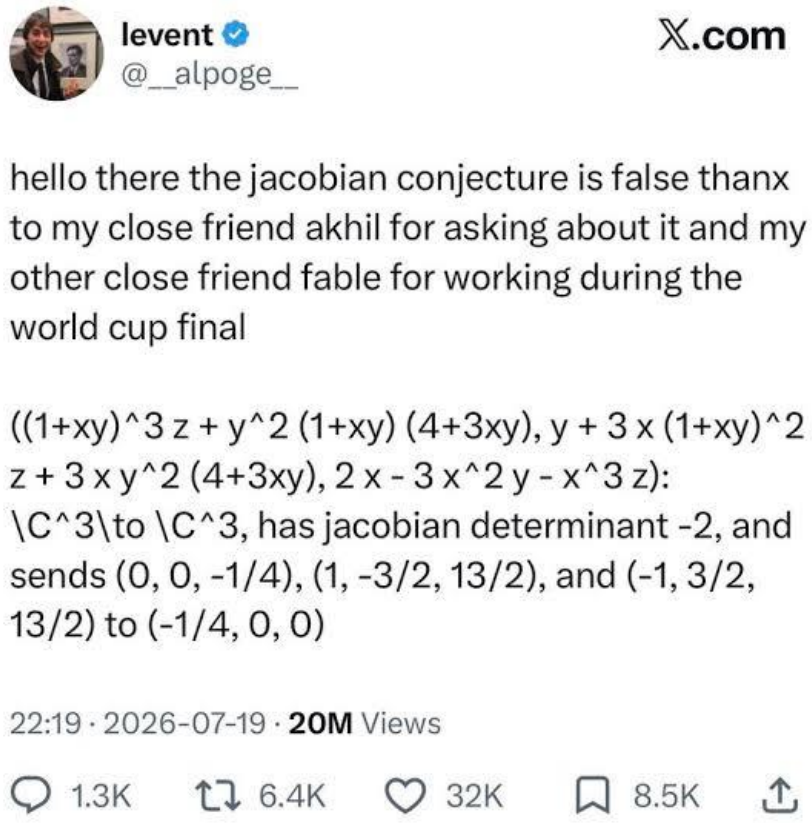
Iteratively improving $R_\phi(x)$, but reward hacking

Online: Getting reward with verifiers

$$R(x) = \begin{cases} 1 \text{ if } x \text{ is correct} \\ 0 \text{ if } x \text{ is wrong} \end{cases}$$

- No reward hacking
- Dense & scalable

- Rewards reasoning: Only correct final answers pay off, so need to discover multi-step strategies.



Inference: Tilting at sampling time

Reward (or classifier-based) guidance: $\nabla \log p_0^*(x) = \nabla \log p_{ref,0}(x) + \frac{1}{\beta} \nabla R(x)$

Train a reward model on noisy versions of x, then add gradient to the unconditional score

Classifier-free guidance: $\nabla \log p_0(x) p_0(y|x)^w = (1-w) \nabla \mathrm{p}_{ref,0}\,(x) + w \nabla p_{ref,0}(x|y)$

Jointly train conditional & unconditional scores and interpolate between them during inference.

Does not actually converge to the true $p^*(x)$

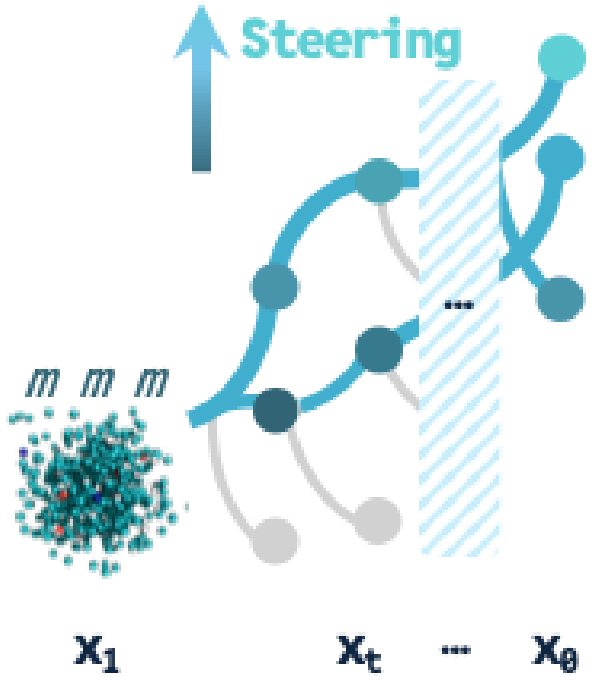
Dhariwal, Prafulla, and Alexander Nichol. "Diffusion models beat gans on image synthesis." *NeurIPS* (2021).
Ho, Jonathan, and Tim Salimans. "Classifier-free diffusion guidance." *arXiv* (2022).

Inference: Fixing the bias with multiple samples

Feymann-Kac Steering

1. Sample N candidates
2. Drift at each step
3. Compute a weight w_i every step which says how good each candidate is
4. Resample the candidates based on w_i

This procedure converges to target $p^*(x)$ as $N \to \infty$!



*Does **not** require access to likelihoods, works under the hood by using Sequential Monte Carlo (SMC)*

Skreta, Marta, et al. "Feynman-kac correctors in diffusion: Annealing, guidance, and product of experts." ICML (2025).
Del Moral, Pierre, Arnaud Doucet, and Ajay Jasra. "Sequential monte carlo samplers." *Statistical Methodology* (2006).

Which one to pick?

Method	Cheap inference	Don't need to train	On or offline	Likelihoods help	Must add model capabilities	Other requirements
Prompt / condition / in-context	✅	✅	—	—	❌	—
SFT	✅	❌	Offline	❌	❌	Curated examples
Best-of-N / RAFT	❌	✅	Offline	❌	❌	Reward fxn
DPO	✅	❌	Offline	✅	❌	Preference pairs
RL: PPO / GRPO	✅	❌	Online	✅	✅	Scalar or verifiable reward
Guidance / FKC / Beam Search	✅ / ❌	✅	—	❌	❌	—

My 2 cents:

Get a verifier & scale, or get a proxy reward & pray

Retrain when you can get a lot of data, otherwise, steer

Outline

Part I. Simulation(-free) Generative Models

Part II. Fine-tuning & Steering

Part III. Applications to Chemistry

Number of Possible Proteins

$>10^{260}$

Atoms in the observable universe 10^{80}

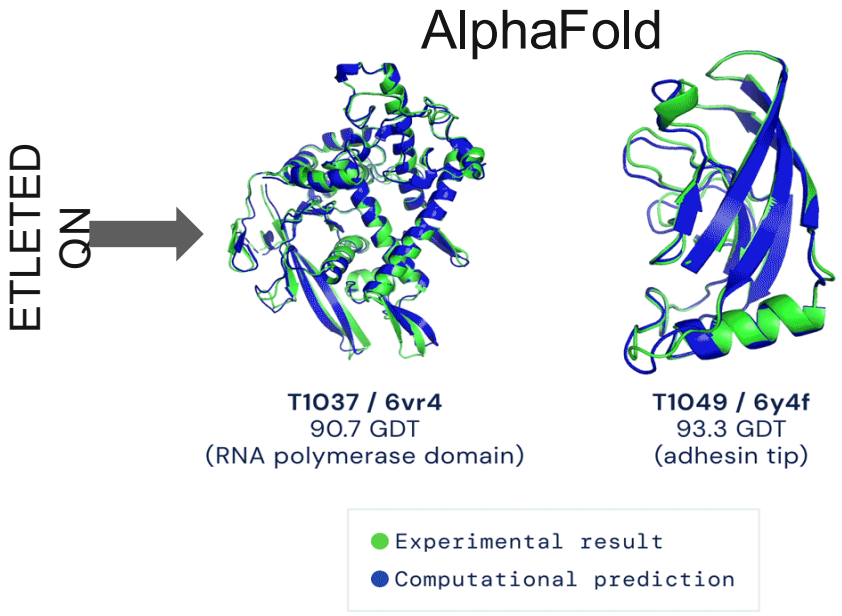
Evolution the tinkerer

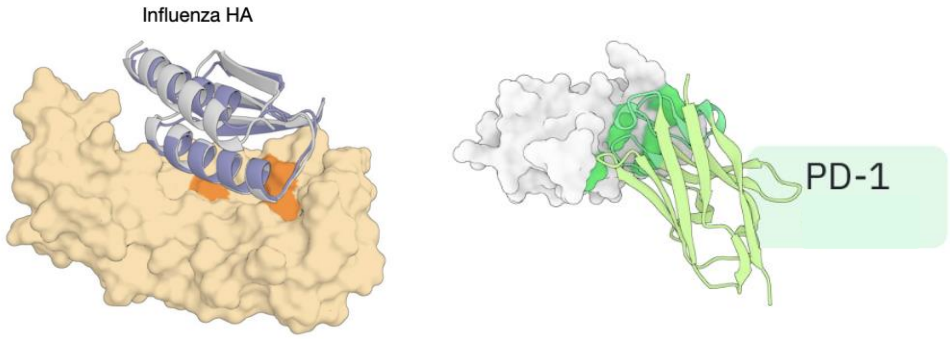
Atoms in the observable universe
10^{80}

- Proteins Nature Has Ever Tried
$\sim 10^{44}$

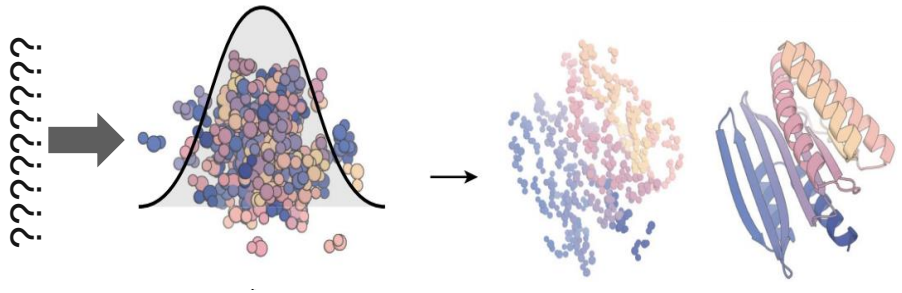
Dryden D.T.F., et al., "How much of protein sequence space has been explored by life on Earth?", *J. R. Soc. Interface* (2008)

Reading & Writing Nature's Language



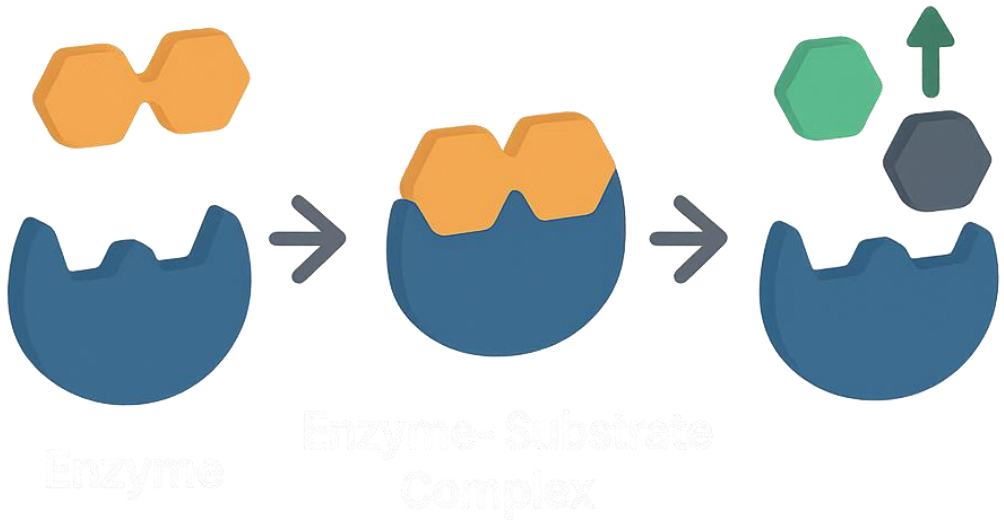


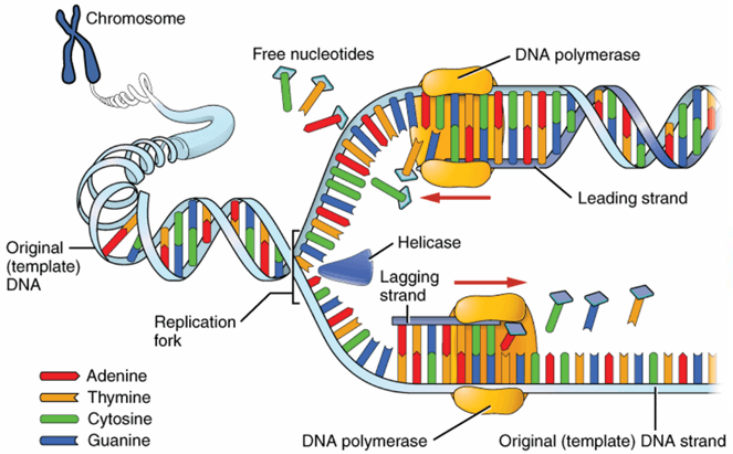
Protein <u>binders</u>
(e.g. miniprotein, nanobodies)

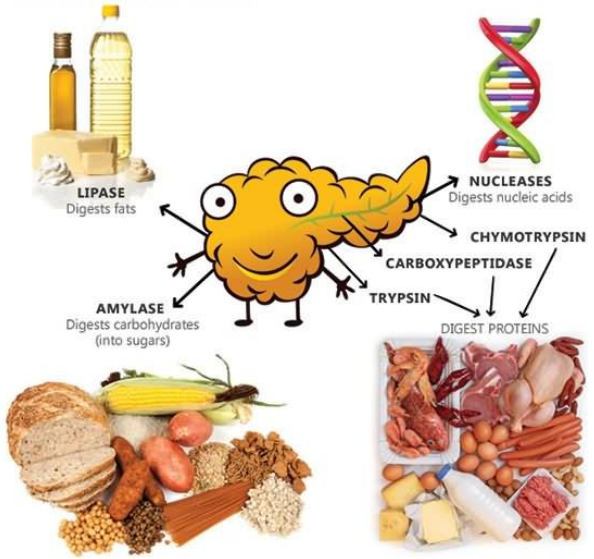


Jumper, John, et al. "Highly accurate protein structure prediction with AlphaFold." *Nature* (2021)
Watson, Joseph L., et al. "De novo design of protein structure and function with RFdiffusion." *Nature* (2023)

What about nature's catalyst: enzymes?





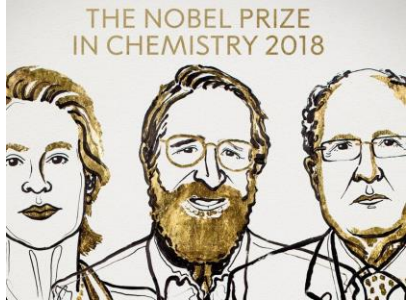


More than nature:

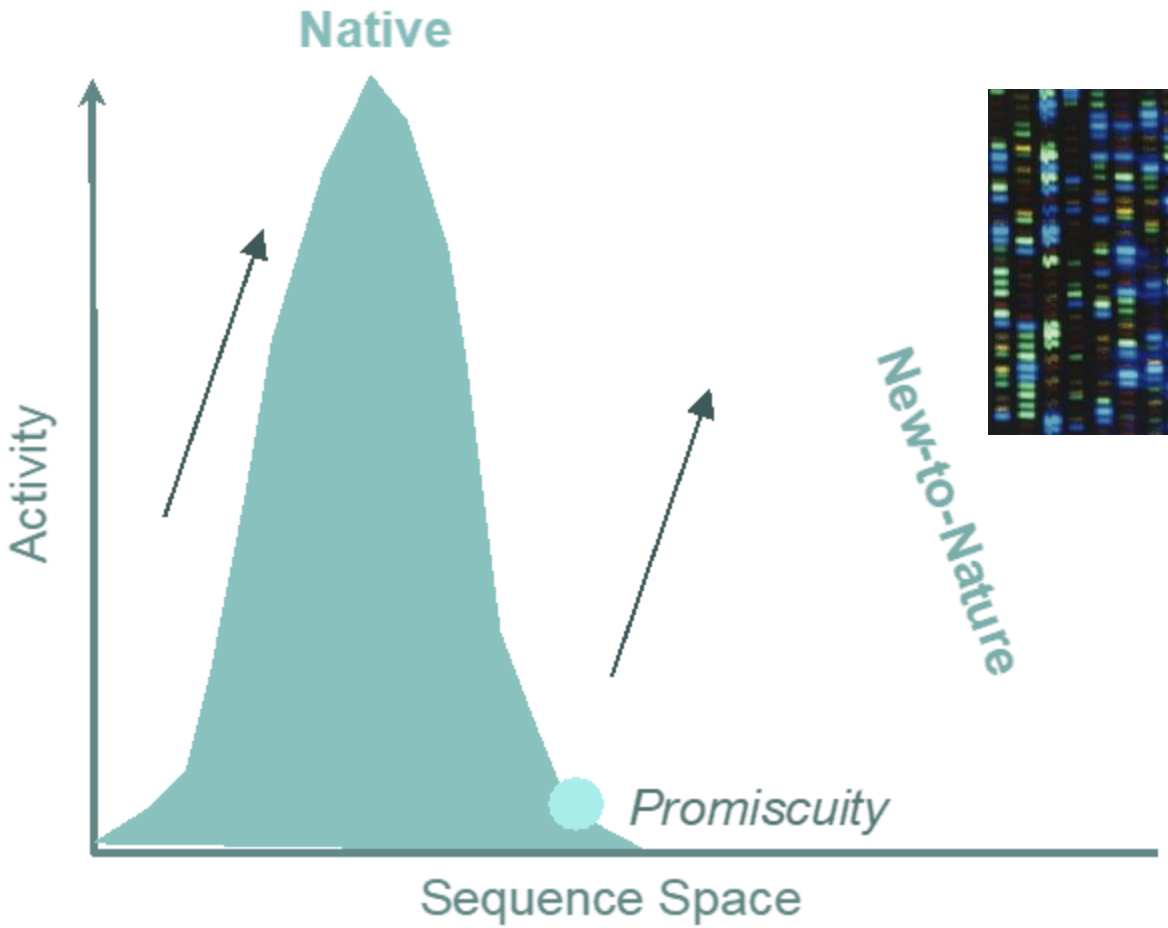
The enzyme pill that makes you not lactose intolerant!

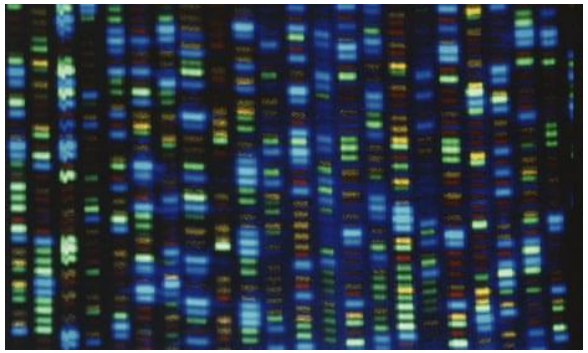
Or, degrade the PFAS in your water?

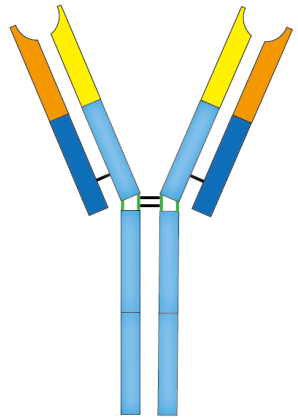
Directed evolution



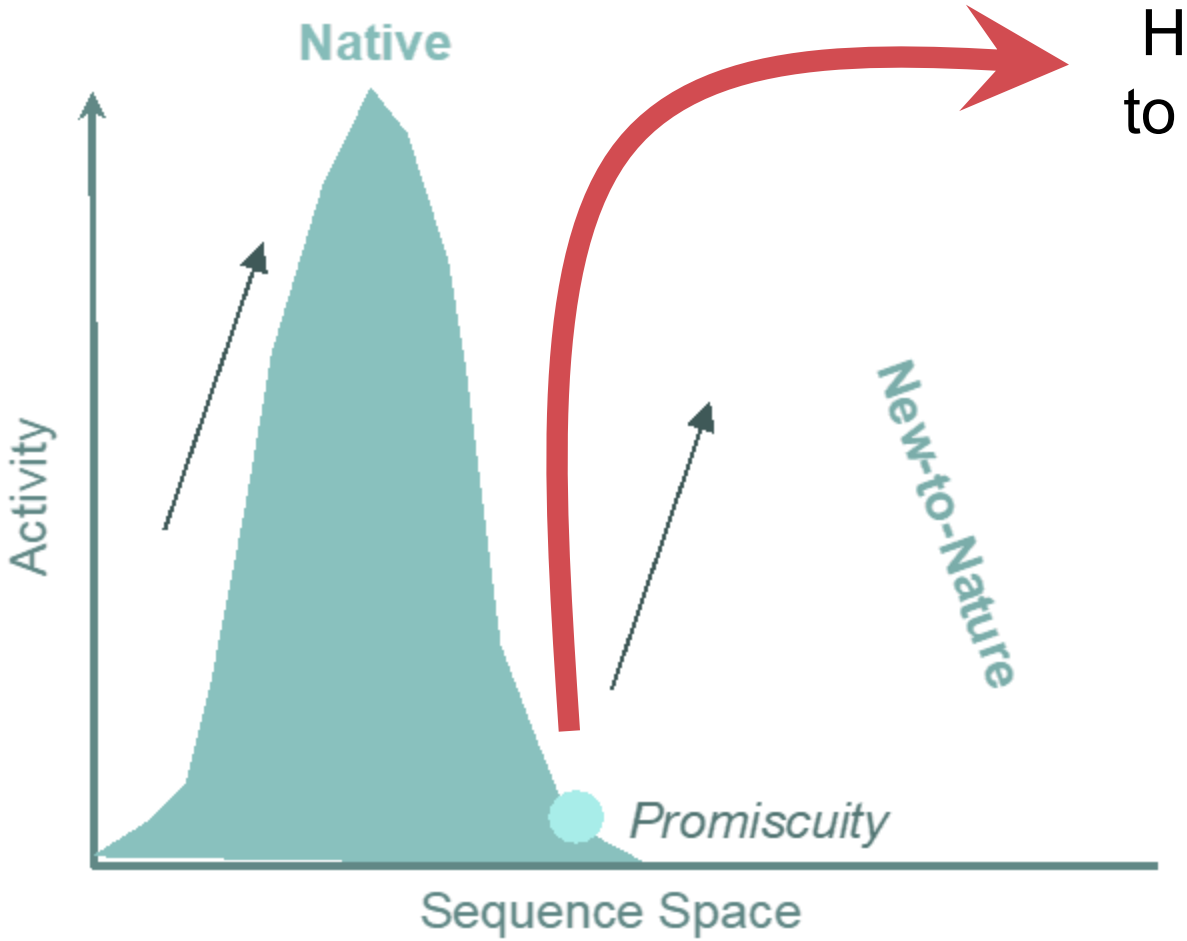
Frances Arnold







The Problem



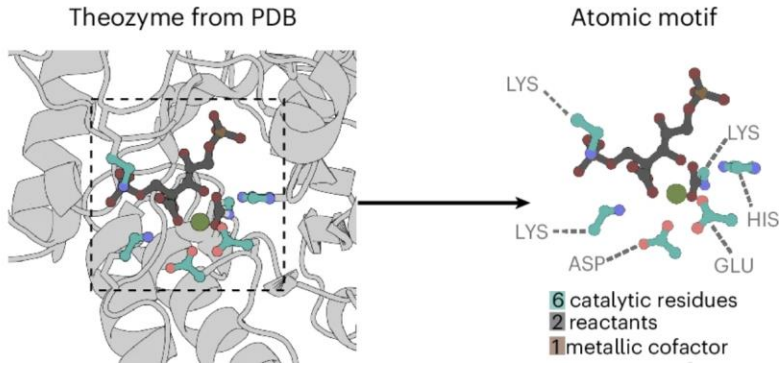
Hard to find!

1. Natural enzymes excel in complicated chemistries, but they are made via evolutionary pressure.

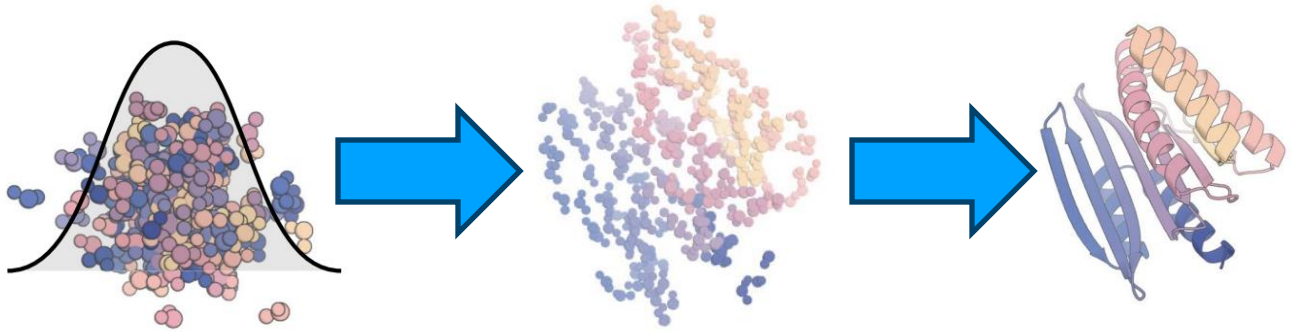
2. Nature has only sampled a very narrow slice of all possible chemistry

Modern enzyme design: starting with a partial solution

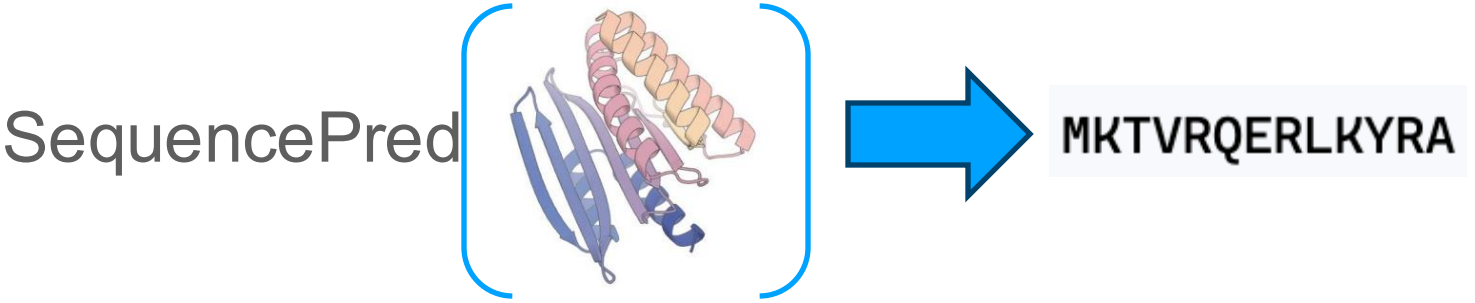
1. Extract a known ‘motif’



2. Generate protein backbone with the motif fixed in space



3. Given a backbone structure, predict a sequence

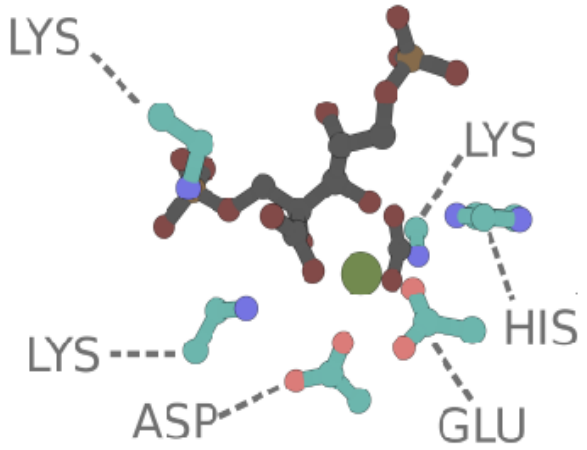


Why shouldn’t we do this

1. No new motif = no new function

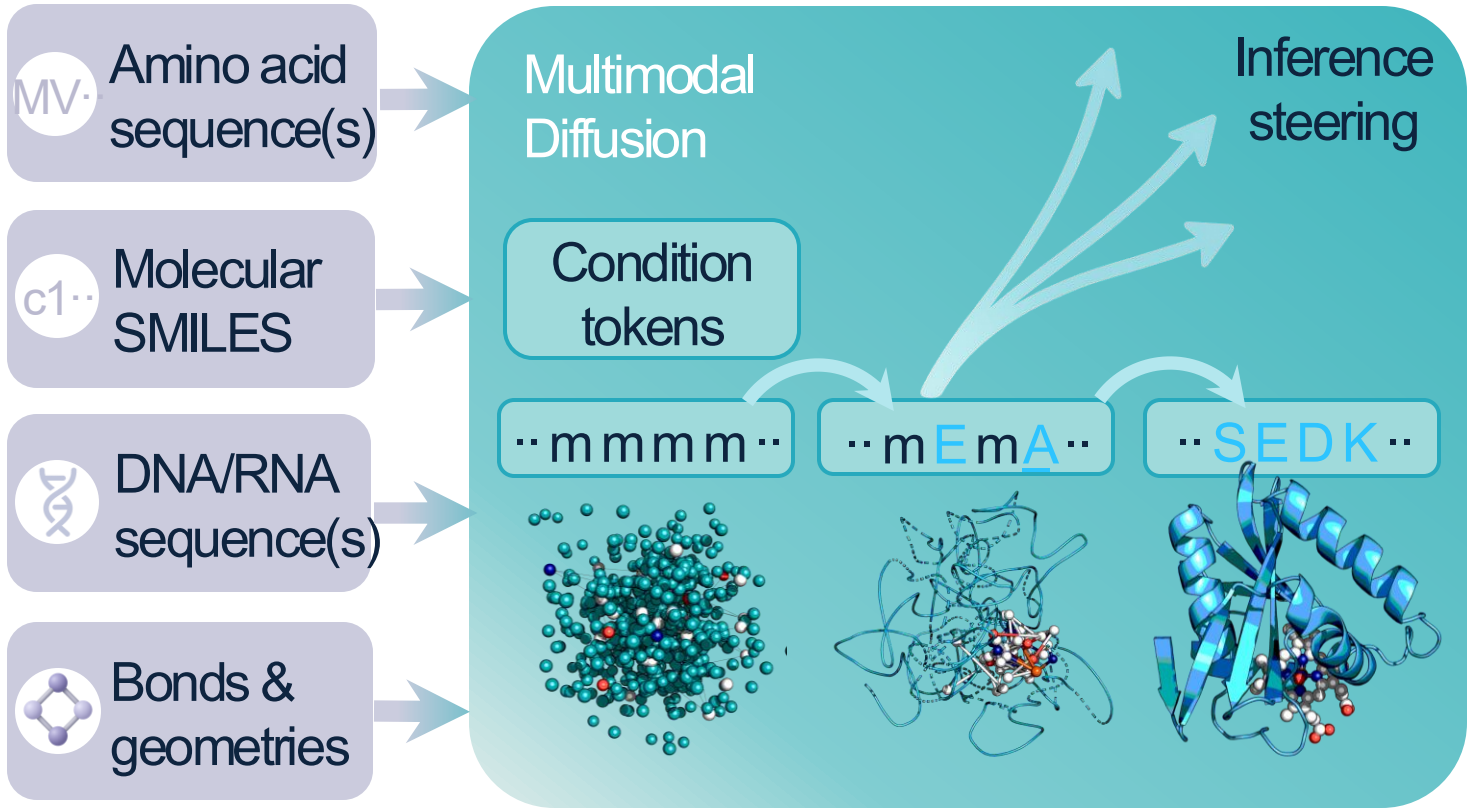
2. Sequence & structure should inform/constrain each other during design.

3. Steer our model based on sequence (cheaper data!)

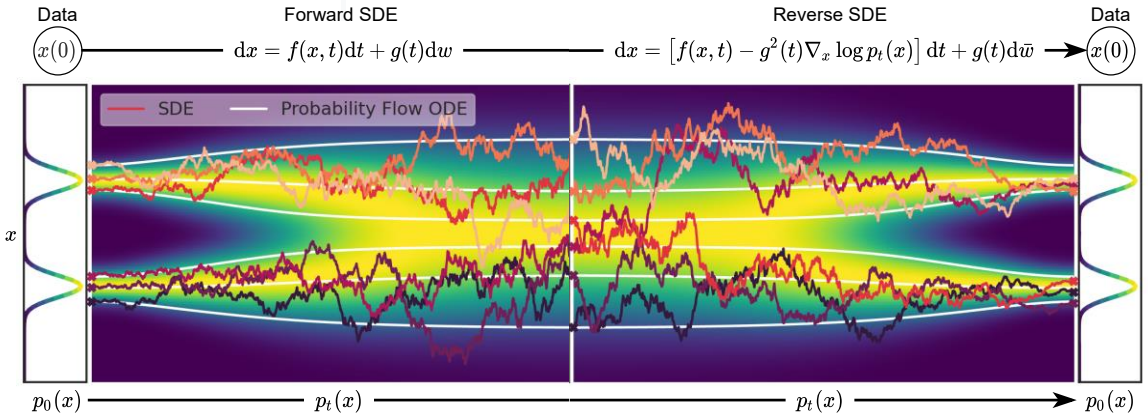


So, what should we do?

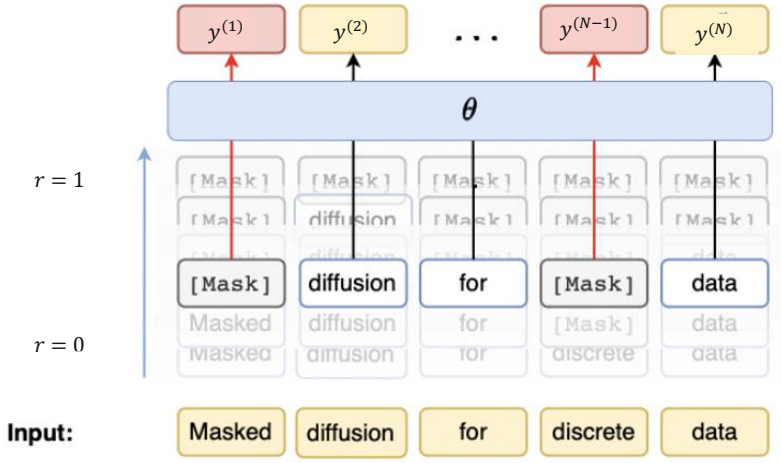
1. Generate protein sequence & structure simultaneously
2. Build informed, new motifs on any biomolecule (ligands, DNA, RNA, TS)
3. Steer the model based on sequence and structure preferences



Building a multimodal model



Continuous diffusion



Discrete diffusion

We need to be able to sample from the joint marginals $p_{t,r}(x_t, y_r)$

Theory says ✅ if we sample $t \sim U[0,1], r \sim U[0,1]$

Practice says ❌ ❌ ❌ ❌ ❌ even if we do everything correct in theory!!!

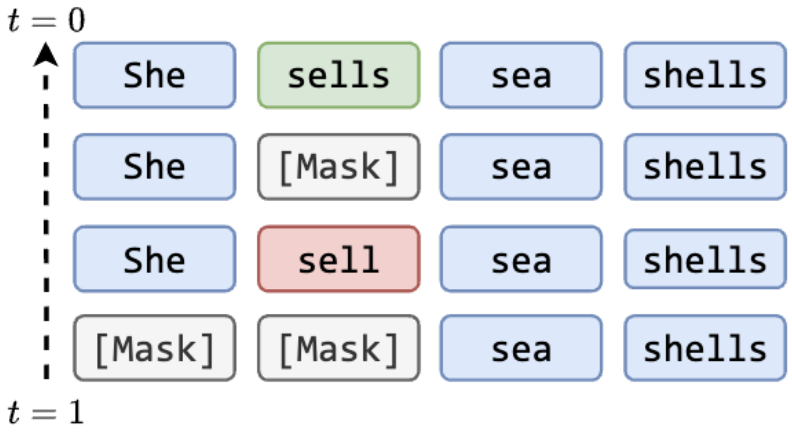
Song, Yang, et al. "Score-based generative modeling through stochastic differential equations." ICLR (2021)
Sahoo, Subham S., et al. "Simple and effective masked diffusion language models." NeurIPS (2024)
Rojas, Kevin, et al. "Diffuse everything: Multimodal diffusion models on arbitrary state spaces." ICML (2025)

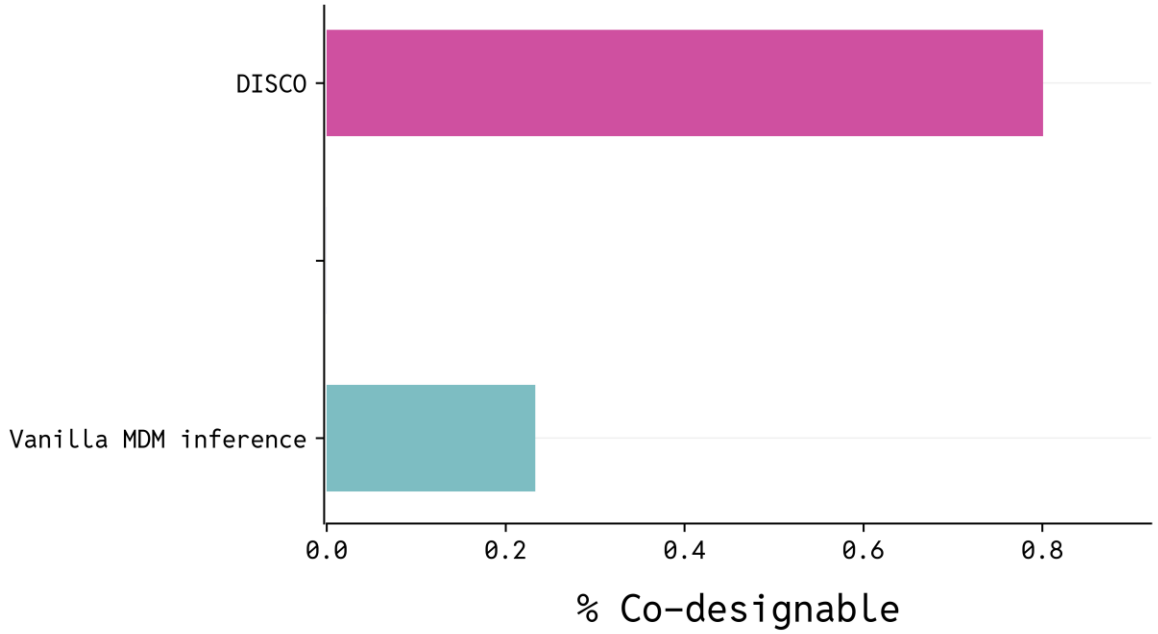
Solving multimodal diffusion

Given a base model, how do we do inference?

Self-correction

Idea: AA prediction might be wrong early, we should let the model correct itself





Peng*, Bezemek*, … "Path planning for masked diffusion model sampling." *arXiv (2025)*
Peng*, Bezemek*, … "Planner Aware Path Learning in Diffusion Language Models Training." *ICLR Oral (2026)*

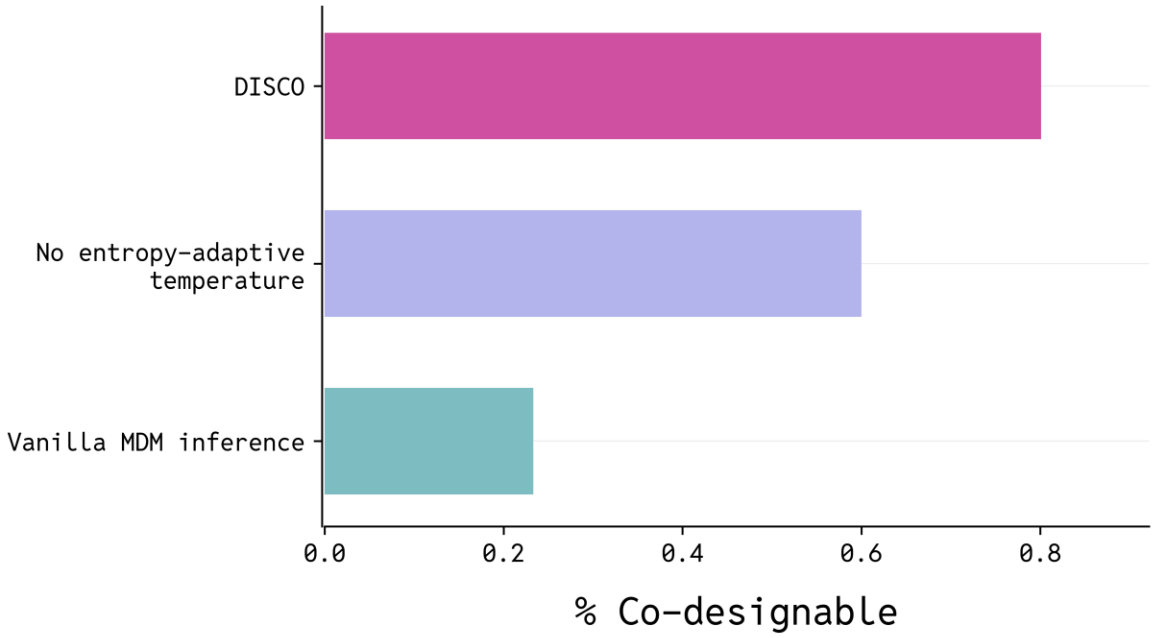
Solving multimodal diffusion

Given a base model, how do we do inference?

Entropy annealed tempering

Idea: Don’t let the model be too confident early when not much structural detail, let it sharpen later

$$\tau_r^{(n)} = \tau\left(1 + \frac{r(H_{max} - H^{(n)})}{H_{max}}\right)$$



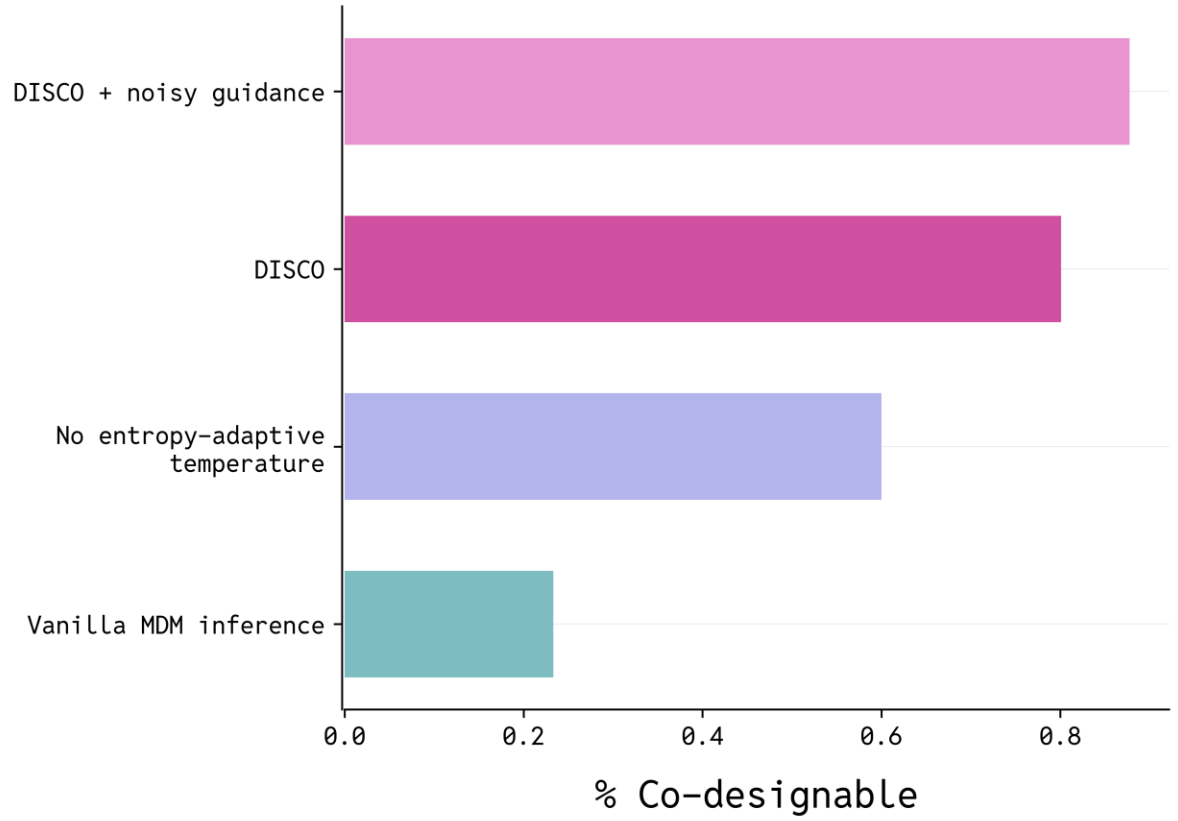
Solving multimodal diffusion

Given a base model, how do we do inference?

Noisy guidance

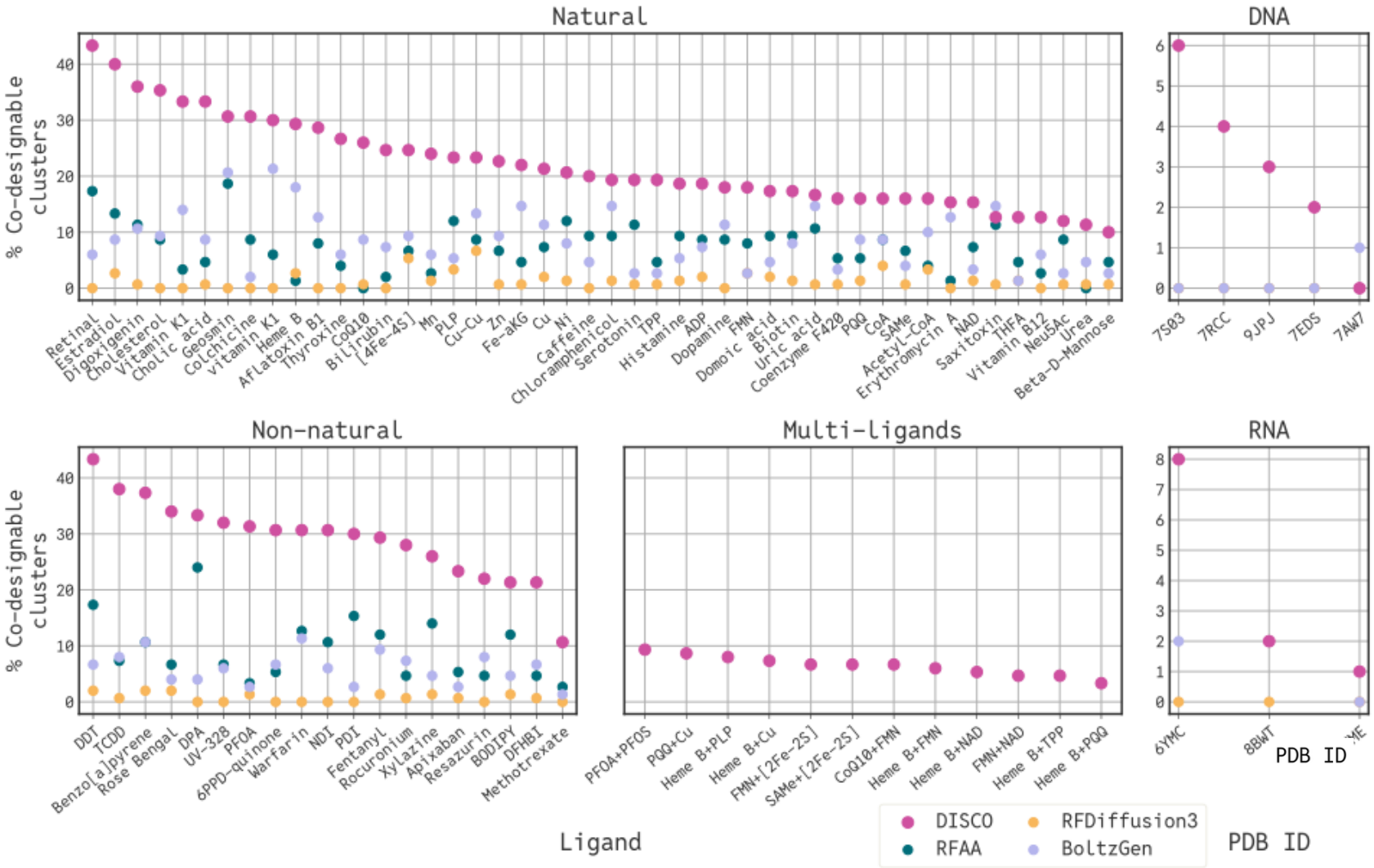
Idea: Condition the model on a noisier version of each modality to sharpen conditional predictions

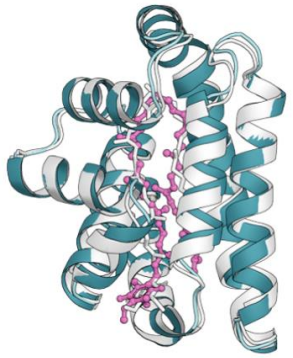
$$\omega s^X_{t,r}(x_t, y_r) + (1-\omega) s^X_{t,r+\Delta}(x_t, y_{r+\Delta})$$
$$\omega s^Y_{t,r}(x_t, y_r) + (1-\omega) s^Y_{t+\Delta,r}(x_{t+\Delta}, y_r)$$



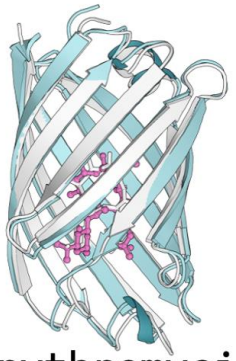
Rojas, Kevin, et al. "Diffuse everything: Multimodal diffusion models on arbitrary state spaces." *ICML* (2025)

DISCO yields SOTA designs

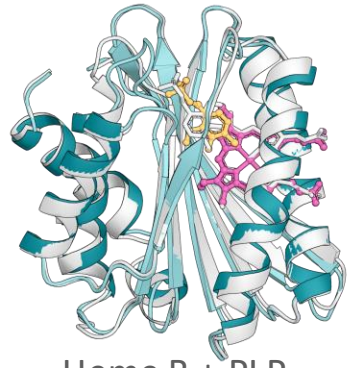




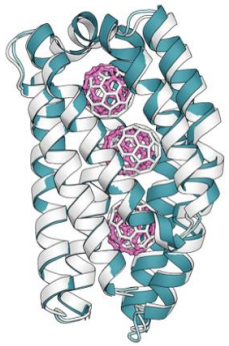
CoQ10



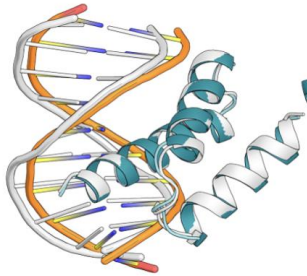
Erythromycin



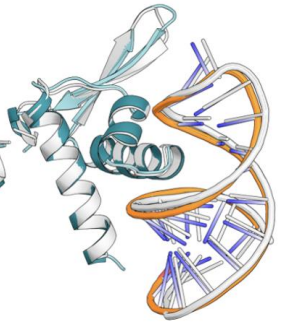
Heme B + PLP



C_{60}×3



DNA: 7RCC



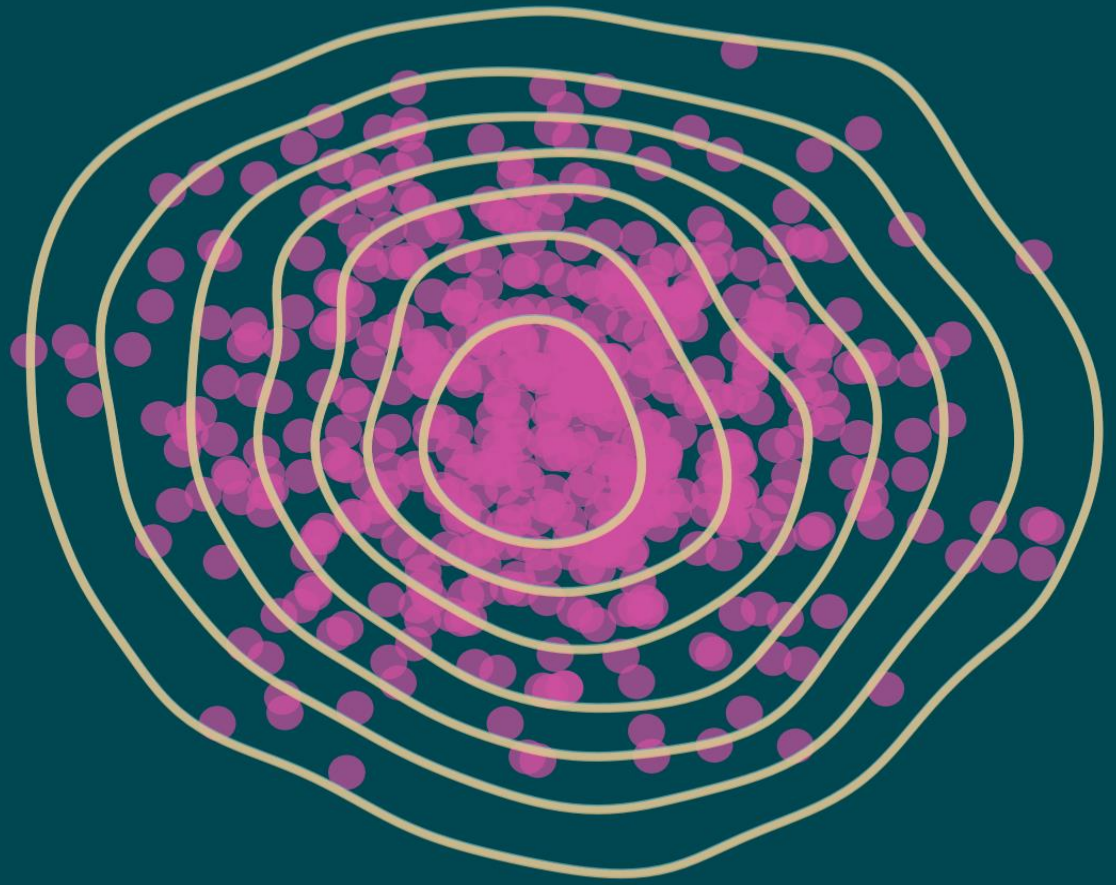
RNA: 6YMC

Krishna, Rohith, et al. "Generalized biomolecular modeling and design with RoseTTAFold All-Atom." *Science* (2024)
Stark, Hannes, et al. "Boltzgen: Toward universal binder design." *bioRxiv* (2025)
Butcher, Jasper, et al. "De novo design of all-atom biomolecular interactions with rfdiffusion3." *bioRxiv* (2025)

Diverse and valid conformers!

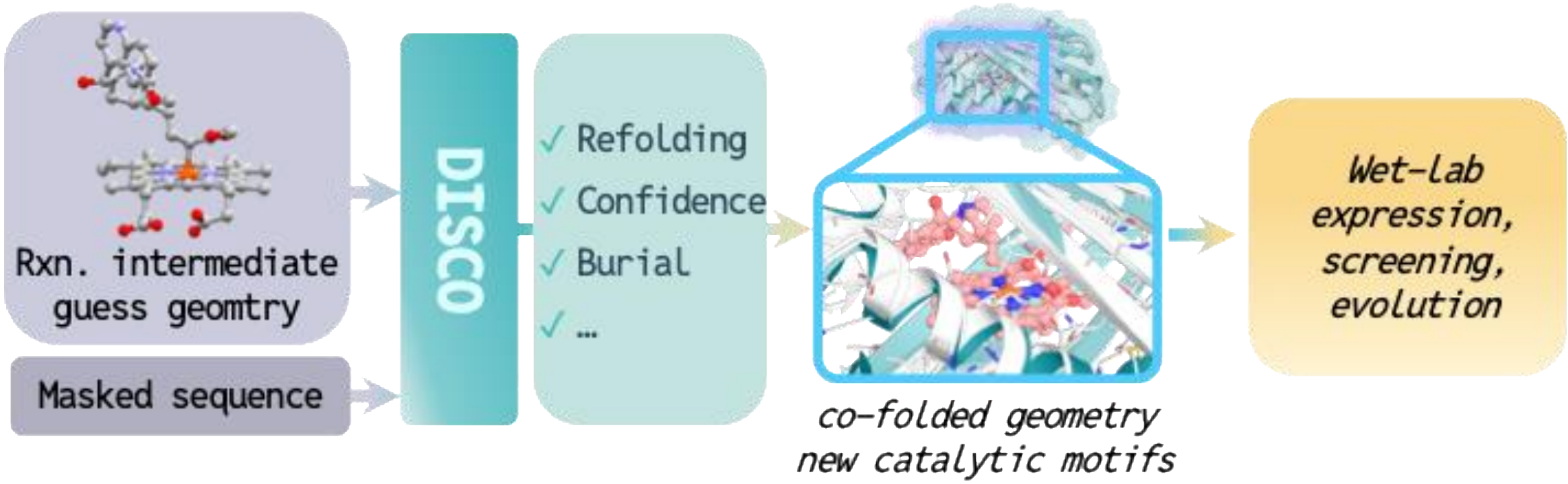
FKC - Specificity Guidance

t=1.000



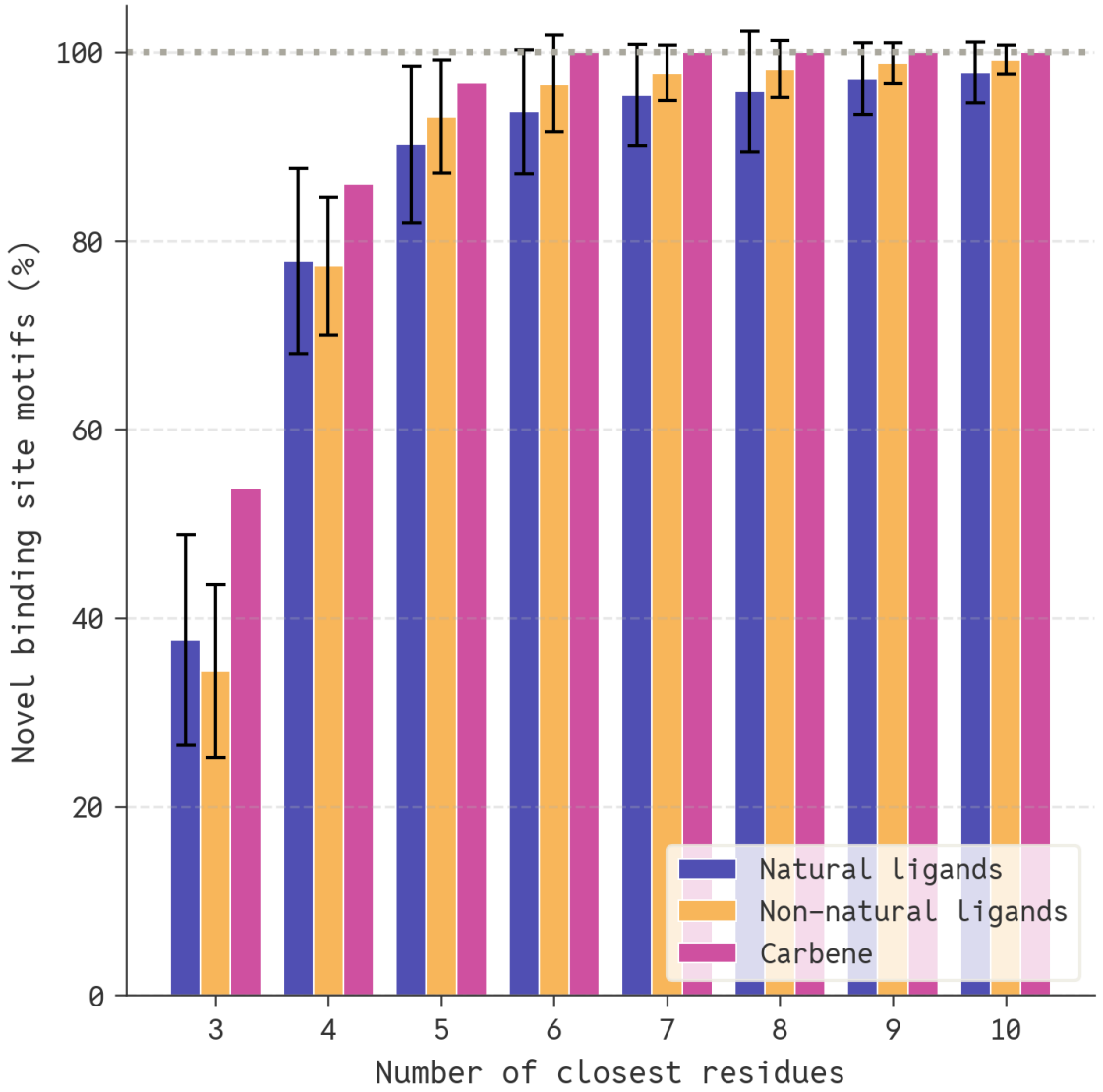
q^{on} q^{off} Generated

DISCO designs enzymes without fixed motifs



Only ~10^4 samples
1-2 orders magnitude fewer than SOTA

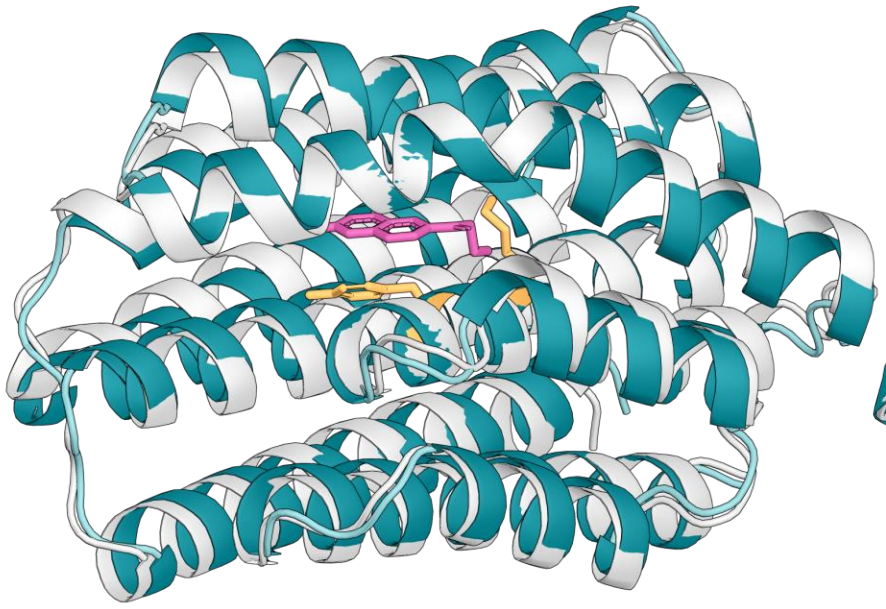
DISCO designs enzymes with new motifs



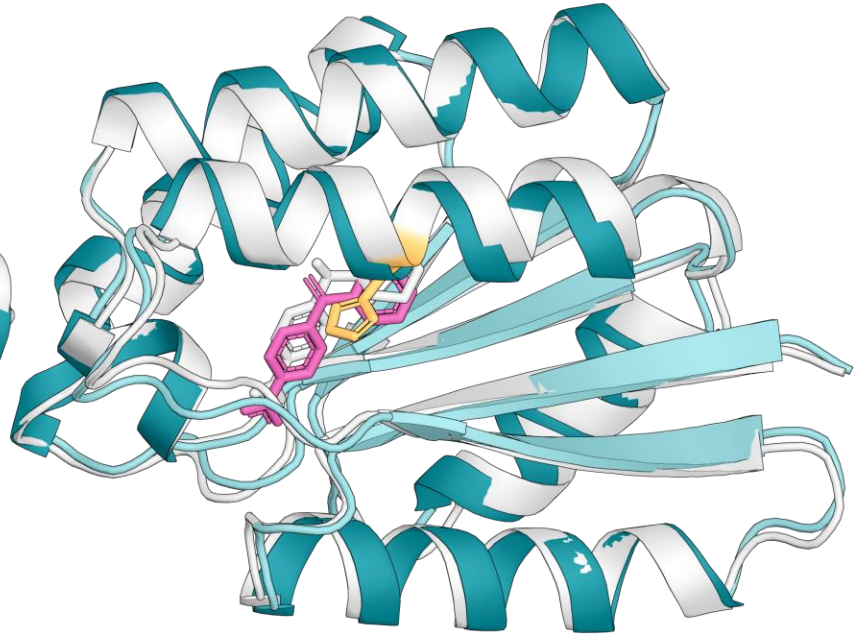
Designing motifs with chemical guidance

Zn metallohydrolase:
His/Asp/Glu coordinate to Zn

+ motif/scaffold novelty -> separate section



Retroaldase:
Lys Nζ → C13
& Tyr present

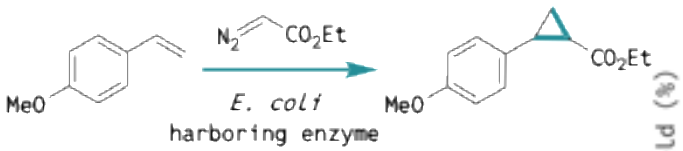


Morita-Baylis-Hillman:
His Nε2 → C13

Theophile 12:15 PM
We can call them discozymes that would sound good
😍 2 1

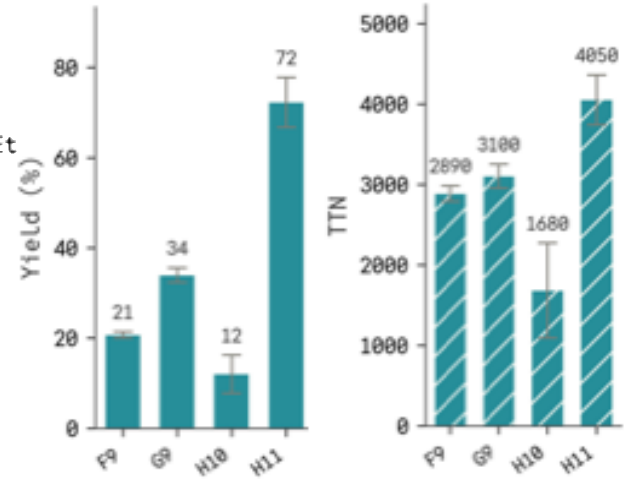
New-to-nature carbene transferases

Alkene cyclopropanation

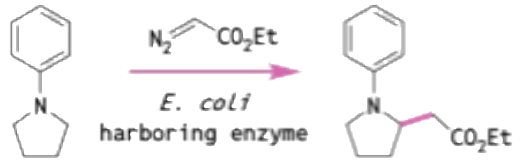


Hemin: 96 TTN
Evolved P450$_{H2\text{-}5\text{-}F10}$: 364 TTN
Heme-binding design C45: 455 TTN
Theozyme-based design PNC2: 630 TTN

Science **2013**, *339*, 307
PNAS, **2020**, 1419
Science **2025**, *388*, 665

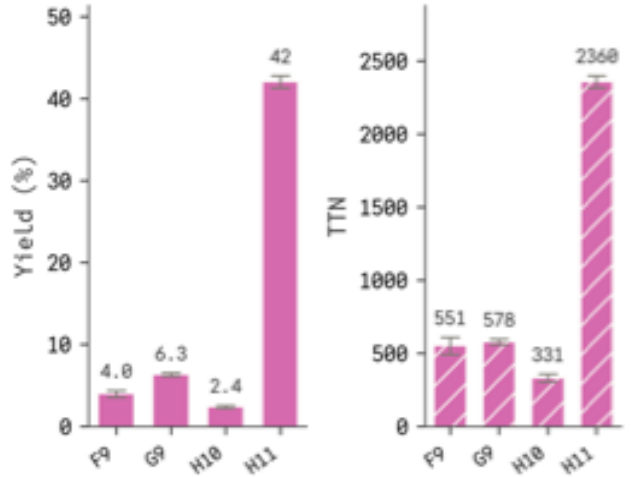


C(sp^3)–H alkylation

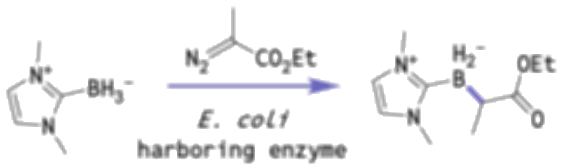


Hemin: ND
P-4 A82L (different substrate): 13 TTN
Evolved P411-CHF: 2030 TTN

Nature **2019**, *565*, 67

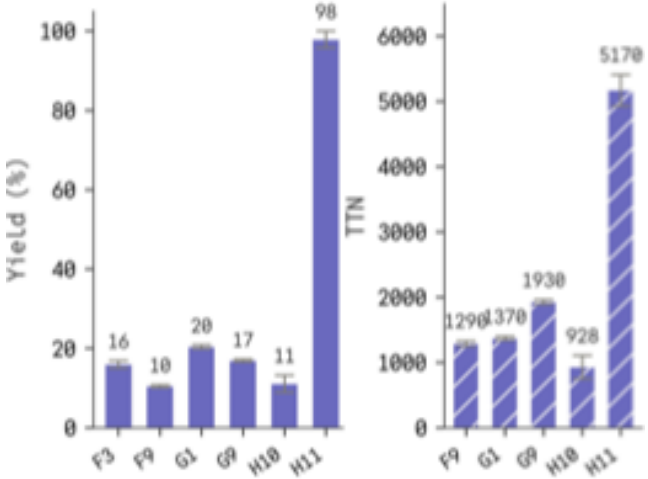


B–H insertion

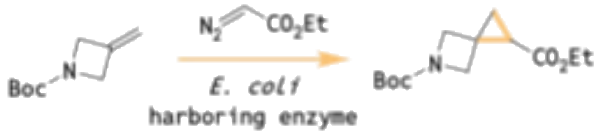


Hemin: 80 TTN
Rma cyt c: 120 TTN
Evolved *Rma* cyt c: 2490 TTN

Nature **2017**, *552*, 132



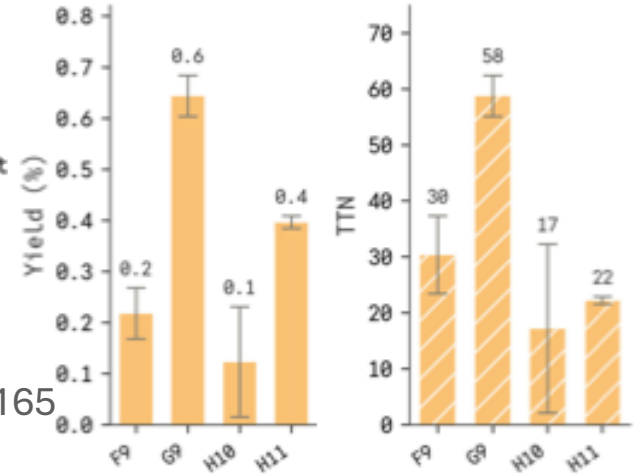
Spirocyclopropanation



Hemin: ND
Evolved ApePgb: 2200 TTN

J. Am. Chem. Soc. **2025**, *147*, 27165

Worse than evolution

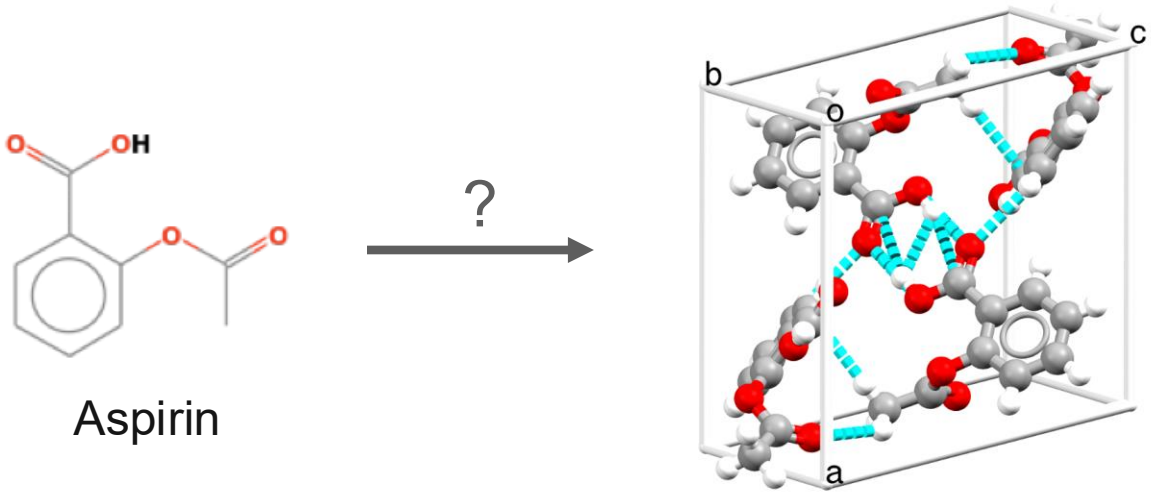


Summary

- Training isn’t everything!
- Solve multimodal diffusion by aligning the modalities at inference time: don’t be overconfident.
- Design enzymes without a catalytic motif, and make it work in the lab!

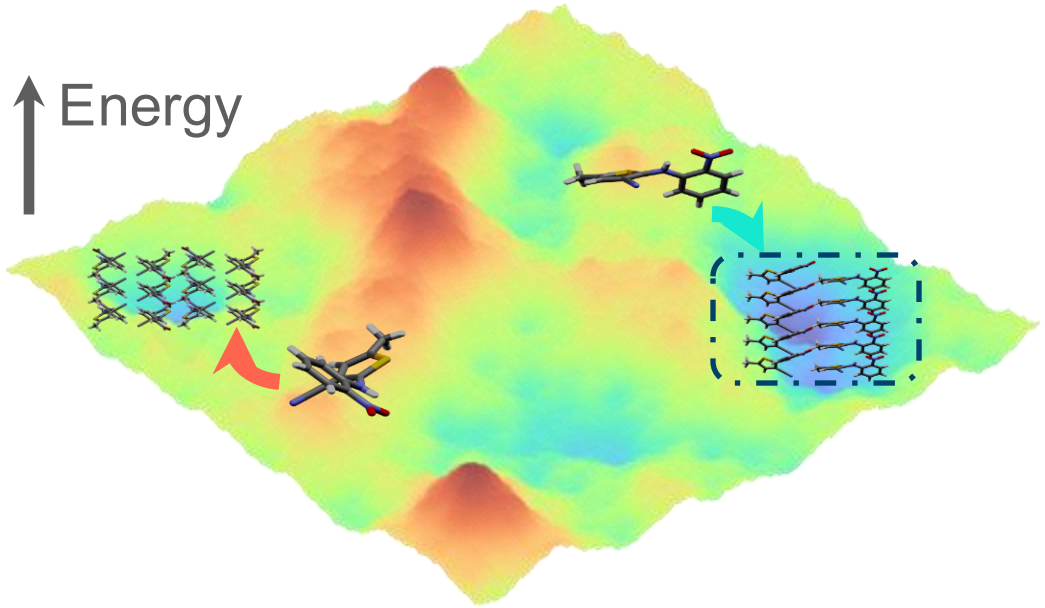
Molecular crystal structure prediction

- Crystal packing directly governs the physical and chemical properties of organic solids.
- Goal: Given a *2D molecular graph*, can we predict its *3D crystal packing*?



Molecular crystal structure prediction

- Crystal packings occupy few local minima in a rugged Gibbs free *energy* landscape, where *kinetics* often dictate which minima are formed
- *Intramolecular* + *intermolecular* interactions.
- More diverse interactions than 20 amino acids

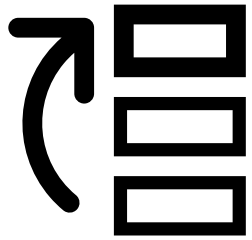


Classical way: via DFT

Step 1: Search

Enumeration, evolutionary algorithms, etc.

Step 2: Rank



Oracle access to energy models (force fields, DFT)

Limitations:

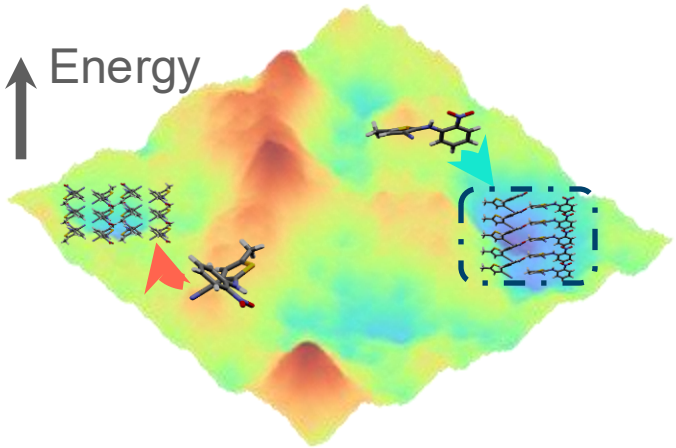
- Doesn’t work on complicated samples
- Require *tens of thousands* of candidate structures per molecule, computationally prohibitively *expensive*

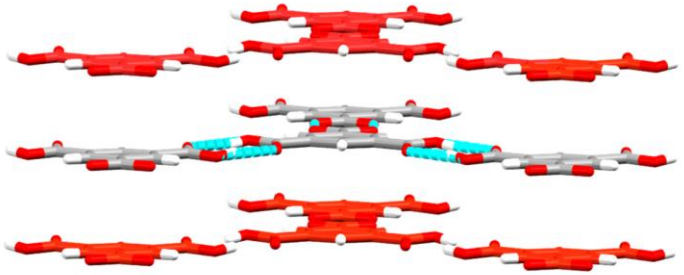
The challenges with CSP

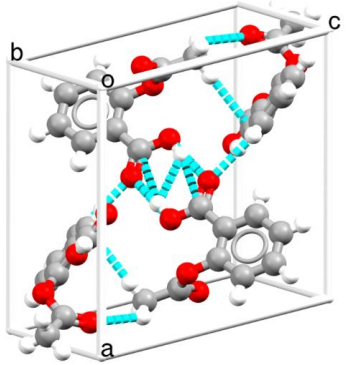
1 *Joint distribution* of thermodynamic and kinetic effects

2 Diverse *long-range* and *weak* molecular interactions

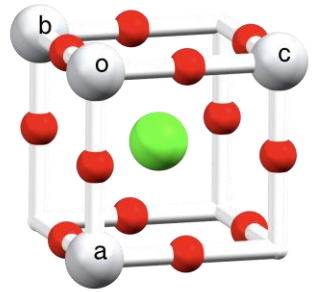
3 Tough symmetry and *ambiguous* Z/Z'







vs

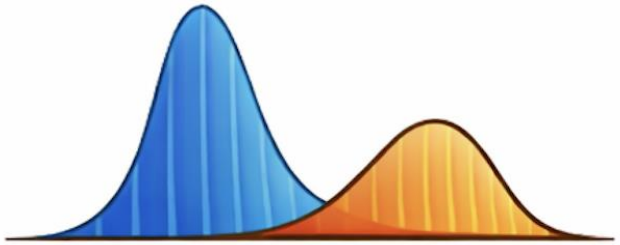


Asprin (Organic)

Perovskite (Inrganic)

OXtal blueprint

1 Joint distribution of thermodynamic and kinetic effects



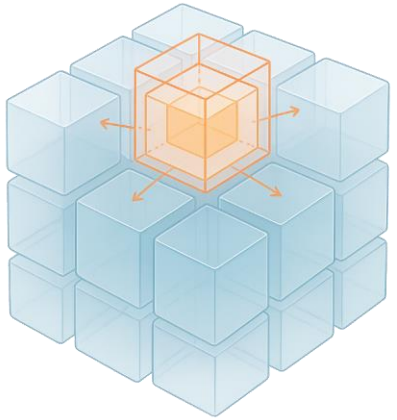
Diffusion Model

2 Diverse long-range and weak molecular interactions

CCDC

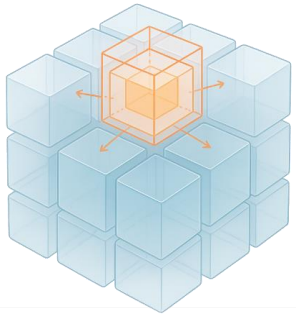
~600K crystal structures

3 Tough symmetry + ambiguous Z/Z'

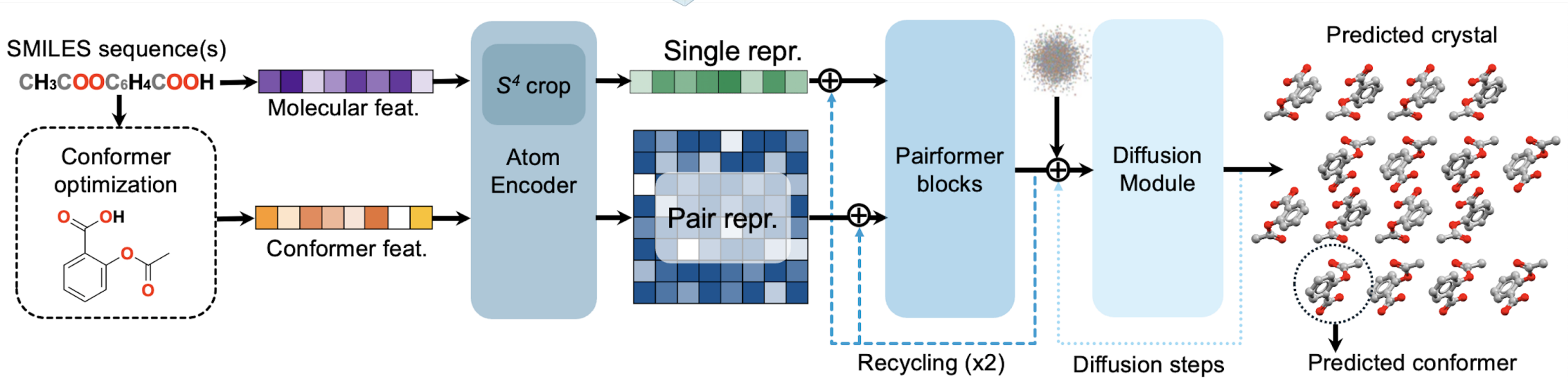


Learn *local interactions* to capture global symmetry

OXtal overview



Schematic of crystal growth



Weighted composite training objective:

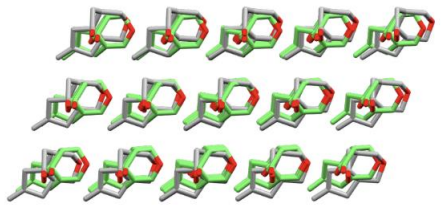
$$\mathcal{L}(\theta) = \mathbb{E}_{t\sim\mathcal{U}(0,1),x_t\sim p_t\left(x_t|x_0^{\text{align}}\right)}\left[\mathcal{L}_{\text{mse}}(\hat{x}_0, x_0^{\text{align}}) + \mathcal{L}_{\text{sLDDT}}(\hat{x}_0, x_0^{\text{align}})\right] + \lambda_{\text{dist}}\mathcal{L}_{\text{dist}}(\hat{d}, d)$$

Global accuracy — Local environment — Pairwise distances

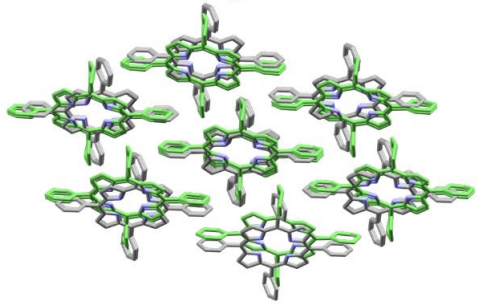
OXtal generates accurate packings

QQQCIG
RMSD$_{15}$=0.55 Å

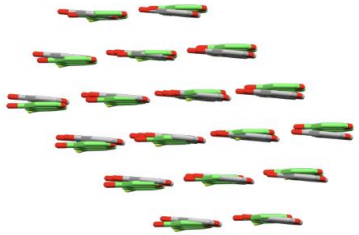
HURYUQ
RMSD$_{15}$=0.58 Å

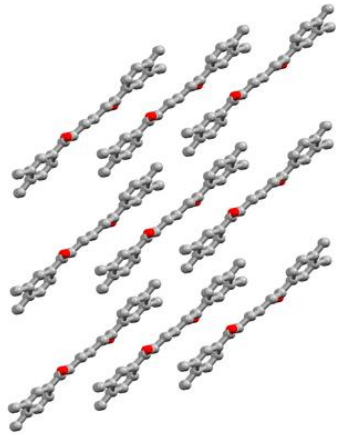


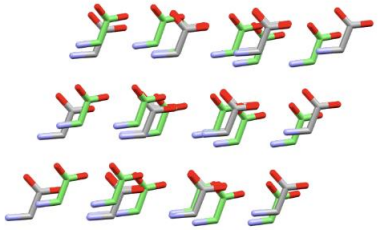
TPHPOR
RMSD$_{15}$=1.23 Å



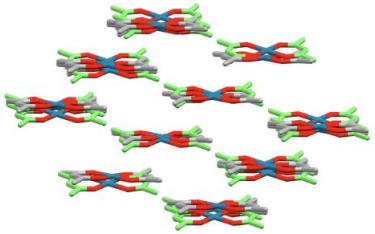
UMIMIO
RMSD$_{15}$=0.55 Å



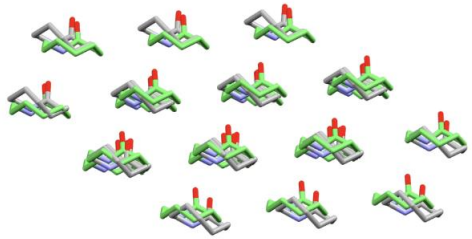




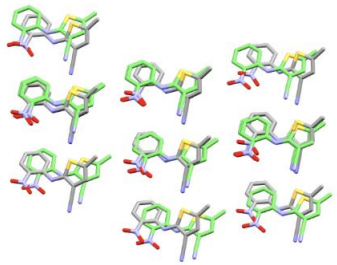
GLYCIN
RMSD$_{15}$=0.74 Å



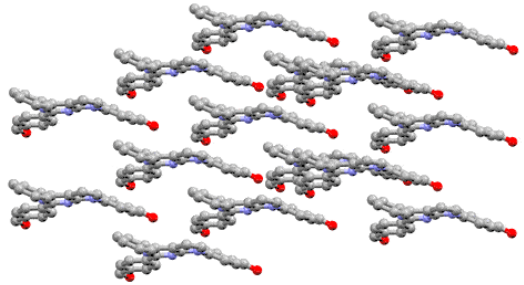
ACACPD
RMSD$_{15}$=1.13 Å



CAPRYL
RMSD$_{15}$=0.73 Å



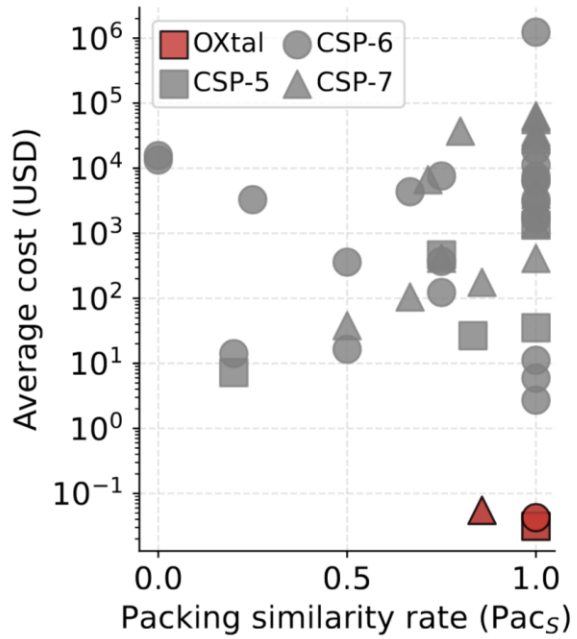
QAXMEH
RMSD$_{15}$=1.24 Å



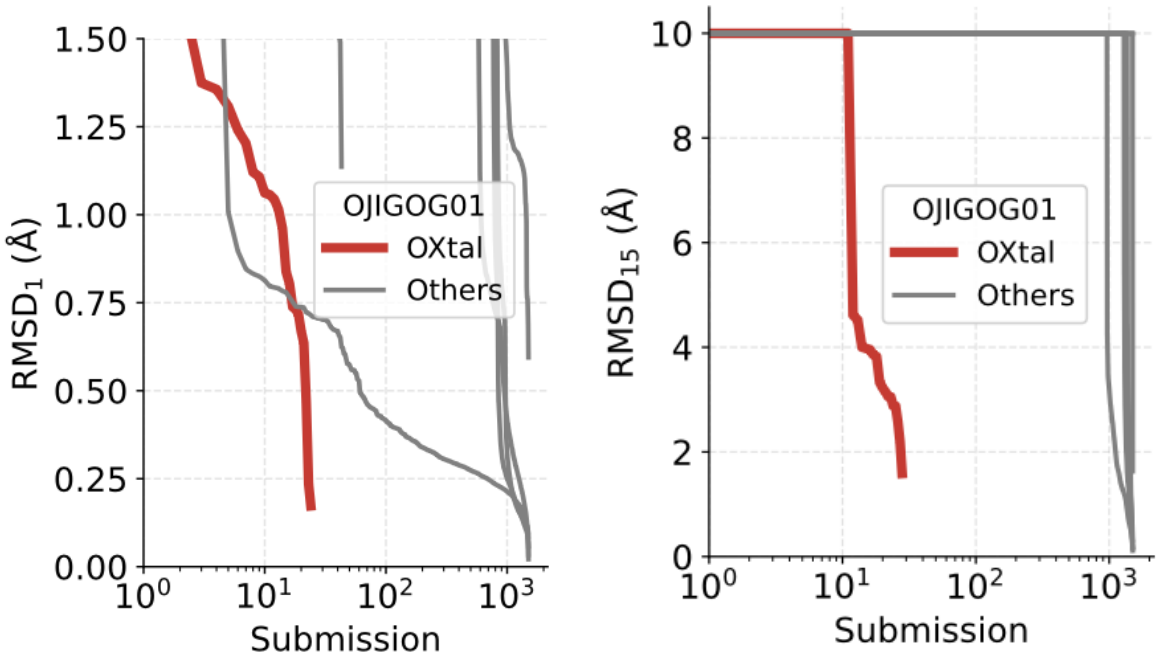
OXtal is competitive to DFT, but much faster

OXtal achieves competitive Pac_S while being *orders of magnitude cheaper* than DFT-based methods

DFT requires *significantly more samples*, suggesting that they are not sampling the correct energy wells



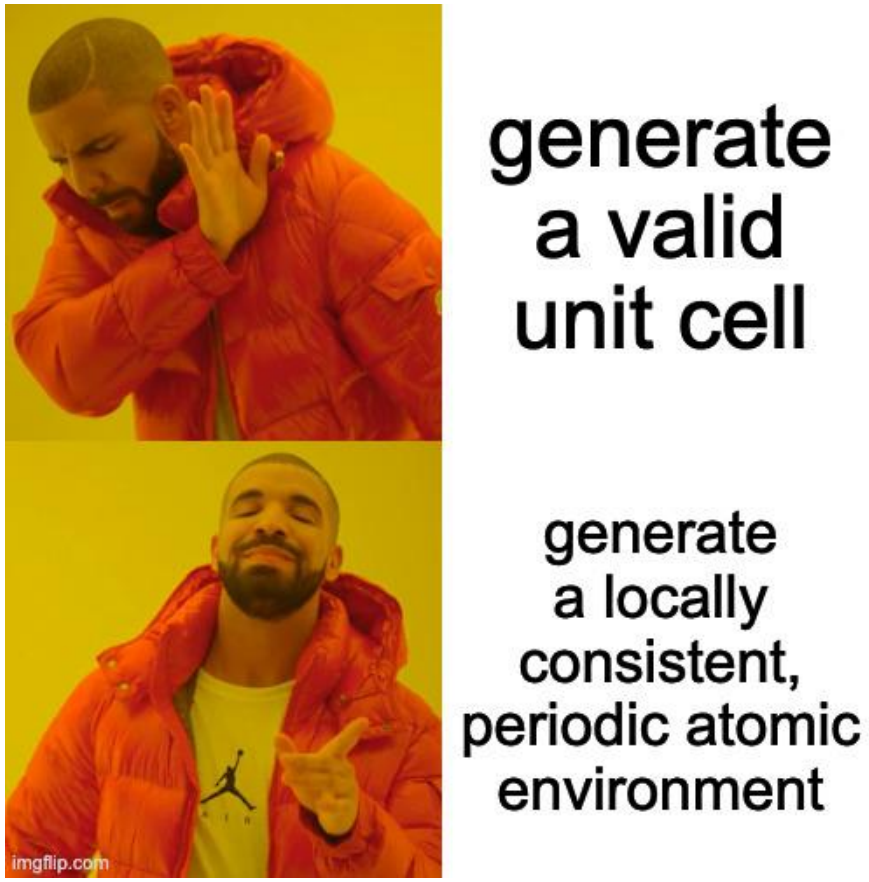
Inference cost of OXtal vs DFT



Sample efficiency (Target XXVIII in CSP7)

Summary

- OXtal is the *first* generative model to successfully scale to molecular crystals
- Reasons: diffusion model scales well when you just do data augmentation! Stable training is the key to success.



Overall summary

- Continuous-time generative models can now train stably in many domains.
- Particularly suited for chemical applications with a lot of reasonably quality data (>10^4) in a huge search space.
- Remaining challenges are:
  - scaling to LLM levels,
  - few step, especially in discrete space,
  - multimodal generation,
  - improving generalization to underlying distribution,
  - how to finetune with noisy rewards.